

آموزش ARM

تهیه و تألیف: محمد مهدی قائمی

mahdighaemi123@gmail.com

codewithmahdi.ir

@codewithmahdi

v1.0.0

فهرست مطالب

فصل ۱: کوچ به سرزمین ۳۲ بیتی؛ معماری ARM و آماده‌سازی آزمایشگاه ۵

۱. داستان ARM؛ یک معماری و نه یک تراشه..... ۵

۲. خانواده‌های ARM؛ سه تفنگدار پردازشی..... ۶

۳. جایگاه STM32 در این بازار..... ۶

۴. تفاوت معماری ۸ بیتی و ۳۲ بیتی؛ راز آدرس‌دهی ۴ گیگابایتی..... ۷

۵. آماده‌سازی آزمایشگاه مجازی..... ۸

فصل ۲: زبان سخت‌افزار؛ نبرد رجیسترها و درایور HAL ۱۰

۱. چالش مدیریت رجیسترهای ۳۲ بیتی..... ۱۰

۲. راه حل مدرن؛ لایه انتزاعی سخت‌افزار یا HAL..... ۱۱

۳. مقایسه عملی؛ شفافیت در برابر ابهام..... ۱۱

۴. ساختار درایور HAL؛ زیر پوست کد چه می‌گذرد؟..... ۱۲

۵. نتیجه گیری؛ انتخاب مسیر مهندسی..... ۱۲

فصل ۳: اولین ضربان؛ پروژه چشمک زن با طعم HAL ۱۳

۱. خداحافظی با کدنویسی تنظیمات؛ جادوی CubeMX..... ۱۳

۲. تنظیمات قلب تپنده؛ پیکربندی کلاک..... ۱۳

۳. تولید کد و ورود به محیط برنامه..... ۱۴

۴. کدنویسی؛ فقط ۲ خط!..... ۱۴

۵. شبیه سازی در پروتئوس..... ۱۵

فصل ۴: حس کردن محیط؛ ورودی ها و دکمه ها ۱۶

۱. خداحافظی با مقاومت های خارجی؛ مفهوم پول-آپ داخلی..... ۱۶

۲. تنظیمات در CubeMX؛ ساده و سریع..... ۱۷

۳. کدنویسی؛ تابع خواندن پین..... ۱۸

۴. راهنمای اتصالات در پروتئوس (طبق تصویر)..... ۱۹

فصل ۵: نمایشگر کاراکتری؛ پورت کردن کتابخانه ۲۰

۱. سخت افزار در پروتئوس؛ سیم کشی ۴ بیتی..... ۲۰

۲. تنظیمات CubeMX..... ۲۱

۳. کدنویسی؛ نوشتن درایور LCD در دل main.c..... ۲۱

۴. تحلیل فنی: تابع sprintf چیست؟..... ۲۳

فصل ۶: توقف ممنوع؛ جادوی وقفه ها ۲۵

۱. سناریوی پروژه؛ دوستی خاله خرسه..... ۲۵

۲. تنظیمات در CubeMX؛ فعال سازی زنگ خطر..... ۲۵

۳. کدنویسی؛ تابع Callback کجاست؟..... ۲۶

۴. تحلیل فنی؛ چه اتفاقی می افتد؟..... ۲۷

۵. بستن مدار در پروتئوس..... ۲۷

فصل ۷: مکالمه دوطرفه؛ ساخت کنسول فرمان با UART ۲۹

۱. تعریف متغیرها؛ ظرفی برای داده‌ها ۲۹
۲. قلب تپنده پروژه؛ تابع Callback ۲۹
۳. شروع برنامه؛ تابع Main ۳۱
۴. شبیه‌سازی نهایی در پروتئوس ۳۱
۵. راهنمای تنظیم دقیق کلاک روی ۱۶ مگاهرتز ۳۲

فصل ۸: مدیریت زمان؛ راه اندازی تایمرهای مستقل ۳۴

۱. مفهوم ریاضی تایمرها ۳۴
۲. تنظیمات در CubeMX ۳۴
۳. کدنویسی؛ مدیریت زمان ۳۵
۴. شبیه‌سازی در پروتئوس ۳۶

فصل ۹: هوشمندسازی دما؛ ساخت ترموستات اتوماتیک ۳۸

۱. سناریوی پروژه ۳۸
۲. سخت‌افزار و اتصالات در پروتئوس ۳۸
۳. تنظیمات نرم‌افزاری در CubeMX ۳۹
۴. کدنویسی پروژه ۴۰
۵. تست نهایی ۴۳

فصل ۱۰: ورودی‌های ماتریسی؛ اتصال ماشین حساب به میکروکنترلر ۴۵

۱. سخت‌افزار و اتصالات ۴۵
۲. تنظیمات نرم‌افزاری در CubeMX ۴۵
۳. کدنویسی پروژه ۴۶

فصل ۱۱: مدیریت همزمان وظایف؛ تقابل CMSIS و Native FreeRTOS ۵۱

۱. تعاریف اولیه و دستگیره‌ها (Handles) ۵۱
۲. ساخت و پیکربندی تسک‌ها (Task Creation) ۵۱

۳. شروع زمان‌بندی (Starting the Scheduler) ۵۲

۴. کد کامل پروژه LED Blinking با دو سبک ۵۲

۵. تحلیل فنی تاخیرها (vTaskDelay vs osDelay) ۵۴

فصل ۱۲: از کدنویسی تا واقعیت؛ اجرای پروژه روی برد ۵۵

قدم اول: کامپایل کردن (مطمئن شو کد سالمه) ۵۵

قدم دوم: چی بخیریم؟ (انتخاب سلاح!) ۵۵

قدم سوم: سیم‌کشی و اتصال پروگرامر ۵۶

قدم چهارم: ریختن برنامه روی برد (Program) ۵۷

فصل ۱: کوچ به سرزمین ۳۲ بیتی؛

معماری ARM و آماده سازی

آزمایشگاه

به دنیای قدرتمند ARM خوش آمدید. اگر تا امروز تصور می کردید که سخت افزار محدود به پردازش های ساده و کند است، اینجا نقطه عطف یادگیری شماست. در این فصل می خواهیم بفهمیم چرا دنیای تکنولوژی به سمت این معماری حرکت کرده است و چطور می توانیم آزمایشگاه مجازی خودمان را برای این تراشه قدرتمند آماده کنیم.

۱. داستان ARM؛ یک معماری و نه یک تراشه

اولین و مهم ترین نکته ای که هست اینکه ARM سازنده تراشه نیست. شرکت ARM در انگلستان تنها نقشه های معماری پردازنده را طراحی می کند و لایسنس یا حق امتیاز آن را به شرکت های دیگر می فروشد.

این بدین معناست که شرکت های بزرگ تولیدکننده نیمه هادی مانند Apple، Samsung و STMicroelectronics نقشه های طراحی هسته پردازشی را از شرکت ARM خریداری می کنند. سپس امکانات جانبی مورد نیاز خود مانند حافظه، وای فای، تایمرها و مبدل های آنالوگ را به آن اضافه کرده و چیپ نهایی را تولید می کنند.

چرا ARM موفق شد؟

معماری ARM بر پایه فلسفه RISC بنا شده است که به معنی دستورات ساده و مصرف انرژی بسیار پایین است. همین ویژگی کلیدی باعث شد تا تقریباً تمام گوشی‌های هوشمند، تبلت‌ها و حالا میکروکنترلرهای مدرن صنعتی از این معماری استفاده کنند.

۲. خانواده‌های ARM؛ سه تفنگدار پردازشی

شرکت ARM برای پوشش دادن تمام نیازهای بازار، پردازنده‌هایش را به سه دسته اصلی تقسیم کرده است که شناخت آن‌ها برای انتخاب قطعه مناسب ضروری است:

سری Cortex-A: حرف A از کلمه Application می‌آید. این سری بسیار قدرتمند است و برای اجرای سیستم‌عامل‌های سنگین مانند اندروید و لینوکس طراحی شده است. مغز متفکر گوشی‌های هوشمند و تبلت‌ها همین سری است.

سری Cortex-R: حرف R از کلمه Real-time می‌آید. این سری در جایی کاربرد دارد که سرعت واکنش به اتفاقات حیاتی است. مانند سیستم ترمز ABS خودرو یا کنترل‌کننده هارد دیسک که تاخیر در آن‌ها پذیرفته نیست.

سری Cortex-M: حرف M از کلمه Microcontroller می‌آید. این همان سری است که ما در این دوره با آن کار داریم. پردازنده‌هایی ارزان، کم‌مصرف اما قدرتمند که برای کنترل سخت‌افزار، رباتیک و اینترنت اشیاء طراحی شده‌اند.

۳. جایگاه STM32 در این بازار

شرکت STMicroelectronics یکی از بزرگترین مشتریان و شرکای تجاری ARM است. این شرکت هسته پردازشی Cortex-M را از ARM تحویل می‌گیرد و اطراف آن امکانات فوق‌العاده‌ای نظیر مبدل‌های آنالوگ دقیق، تایمرهای پیشرفته و پورت‌های ارتباطی متنوع می‌چیند و نام تجاری آن را STM32 می‌گذارد.

تراشه‌ای که ما در این دوره به عنوان مرجع استفاده می‌کنیم STM32F401RE نام دارد. این تراشه از هسته Cortex-M4 استفاده می‌کند و دارای واحد محاسبات اعشاری سخت‌افزاری است که قدرت پردازشی بسیار بالایی برای کارهای مهندسی فراهم می‌کند.

۴. تفاوت معماری ۸ بیتی و ۳۲ بیتی؛ راز آدرس‌دهی ۴ گیگابایتی

بیایید معماری‌های قدیمی ۸ بیتی مانند AVR را با معماری ۳۲ بیتی ARM مقایسه کنیم تا دلیل برتری مطلق ARM روشن شود.

محدودیت دنیای ۸ بیتی:

در یک معماری ۸ بیتی تمام رجیسترها و متغیرهای اصلی ۸ بیت عرض دارند. یعنی نهایتاً می‌توانند عدد ۲۵۵ را در خود جا دهند. اگر بخواهید دو عدد بزرگ مانند ۱۰۰۰ را با هم جمع کنید، پردازنده باید در چندین مرحله و با صرف زمان زیاد این کار را انجام دهد چون ظرفش کوچک است.

قدرت دنیای ۳۲ بیتی:

در ARM رجیسترها ۳۲ بیتی هستند. یعنی می‌توانند اعدادی تا حدود ۴ میلیارد را تنها در یک سیکل ساعت پردازش کنند. این یعنی سرعت بسیار بالا در محاسبات ریاضی و پردازش سیگنال.

جادوی فضای آدرس‌دهی:

مهم‌ترین تفاوت در باس آدرس یا Address Bus است. پردازنده برای دسترسی به هر خانه حافظه باید آدرس آن را صدا بزند.

در یک سیستم ۳۲ بیتی، پردازنده می‌تواند ۲ به توان ۳۲ خانه حافظه را آدرس‌دهی کند.

اگر محاسبه کنید متوجه می‌شوید که ۲ به توان ۳۲ بایت دقیقاً برابر است با ۴ گیگابایت.

$$2^{32} \text{ Bytes} = 4,294,967,296 \text{ Bytes} = 4 \text{ GB}$$

نتیجه چیست؟

این یعنی در STM32 تمام حافظه فلش، حافظه رم و تمام رجیسترهای سخت‌افزاری، همگی در یک نقشه حافظه خطی و یکپارچه ۴ گیگابایتی قرار دارند. پردازنده به تمام اجزای سیستم دسترسی مستقیم دارد و دیگر نیازی به تکنیک‌های قدیمی مانند بانک‌بندی حافظه نیست. این ویژگی کار برنامه‌نویس را بسیار ساده و کد را بهینه می‌کند.

۵. آماده‌سازی آزمایشگاه مجازی

برای شروع کار ما به دو ابزار نرم‌افزاری قدرتمند نیاز داریم. یکی برای نوشتن کد و دیگری برای تست مدار قبل از ساخت واقعی.

الف: نصب محیط توسعه STM32CubeIDE

برخلاف میکروکنترلرهای قدیمی که باید همه چیز را از صفر می‌نوشتیم، شرکت ST محیطی حرفه‌ای بر پایه Eclipse ارائه کرده است.

این نرم‌افزار یک ویژگی جادویی به نام CubeMX در دل خود دارد. CubeMX یک محیط گرافیکی است که شکل میکروکنترلر را به شما نشان می‌دهد. شما روی پایه‌ها کلیک می‌کنید و می‌گویید این پایه ورودی باشد و آن یکی خروجی. سپس نرم‌افزار به صورت خودکار هزاران خط کد اولیه و تنظیمات پیچیده را برای شما تولید می‌کند تا شما فقط روی منطق اصلی برنامه تمرکز کنید.

مراحل نصب:

کافیست نام STM32CubeIDE را جستجو کرده و آخرین نسخه را نصب کنید. این نرم‌افزار کاملاً رایگان است و تمام ابزارهای لازم از جمله کامپایلر و دیباگر را همراه خود دارد.

ب: آماده‌سازی شبیه‌ساز پروتوس

از آنجایی که بردهای آموزشی STM32 ممکن است همیشه در دسترس نباشند، ما از نرم‌افزار Proteus برای شبیه‌سازی استفاده می‌کنیم. نسخه ۸.۹ به بالا از سری STM32F4 پشتیبانی می‌کند.

پس از نصب پروتئوس، باید مطمئن شوید که کتابخانه‌های سری F401 نصب هستند تا بتوانید میکروکنترلر را در لیست قطعات پیدا کنید.

ج: پل ارتباطی؛ ساخت فایل Hex

برای اینکه کدی که در CubeIDE نوشته‌اید در پروتئوس اجرا شود، باید فایل خروجی با پسوند hex تولید کنید. به صورت پیش‌فرض این قابلیت در CubeIDE غیرفعال است.

روش فعال‌سازی تولید: Hex

۱. در محیط CubeIDE روی نام پروژه راست کلیک کنید و گزینه Properties را انتخاب کنید.

۲. به بخش C/C++ Build و سپس Settings بروید.

۳. در سربرگ Tool Settings گزینه MCU Post build outputs را پیدا کنید.

۴. تیک گزینه Convert to Intel Hex file را فعال کنید.

حالا هر بار که دکمه Build یا چکش را بزنید، در پوشه Debug پروژه شما یک فایل Hex ساخته می‌شود که می‌توانید آن را به پروتئوس بدهید تا مدار شما زنده شود.

با تکمیل این مراحل، شما یک آزمایشگاه کامل مهندسی روی سیستم خود دارید و آماده‌اید تا اولین پروژه ۳۲ بیتی خود را آغاز کنید.

فصل ۲: زبان سخت افزار؛ نبرد رجیسترها و درایور HAL

در معماری های ۳۲ بیتی مدرن مانند ARM با سخت افزاری روبرو هستیم که هزاران امکانات و تنظیمات مختلف دارد. مدیریت این حجم از پیچیدگی نیازمند یک استراتژی نرم افزاری هوشمندانه است. در این فصل بررسی می کنیم که چرا مهندسان حرفه ای به جای درگیر شدن مستقیم با بیت ها و آدرس های حافظه، از لایه های انتزاعی یا Abstraction استفاده می کنند.

۱. چالش مدیریت رجیسترهای ۳۲ بیتی

در یک میکروکنترلر پیشرفته هر بخش سخت افزاری توسط تعدادی رجیستر کنترل می شود. در معماری ۳۲ بیتی هر رجیستر ۳۲ خانه حافظه یا بیت دارد. این یعنی برای تنظیم یک بخش ساده باید حواسمان به ۳۲ کلید مختلف باشد.

مشکل زمانی بزرگتر می شود که بدانیم برای راه اندازی یک امکان ساده مانند یک پین خروجی، دیگر یک رجیستر کافی نیست. معماری ARM برای انعطاف پذیری بیشتر، تنظیمات را بین چندین رجیستر تقسیم کرده است:

۱. رجیستر کلاک: **RCC** ابتدا باید برق آن بخش را وصل کنید.
۲. رجیستر **MODER** باید تعیین کنید پین ورودی است یا خروجی.
۳. رجیستر **OTYPER** باید نوع ترانزیستور خروجی را تعیین کنید.
۴. رجیستر **OSPEEDR** باید سرعت سوئیچینگ پین را تنظیم کنید.
۵. رجیستر **PUPDR** باید وضعیت مقاومت های داخلی را مشخص کنید.

۶. رجیستر ODR: در نهایت داده را روی پین بنویسید.

اگر بخواهیم به روش سنتی و با مقداردهی مستقیم به آدرس‌های حافظه این کار را انجام دهیم، کدی شبیه به این خواهیم داشت:

```
1. // Register Level Code - The Hard Way
2. RCC->AHB1ENR |= 0x00000001; // Enable Clock for Port A
3. GPIOA->MODER |= 0x00000400; // Set Pin 5 to Output Mode
4. GPIOA->ODR   |= 0x00000020; // Set Pin 5 High
```

مشکل این روش کاملاً واضح است. این اعداد هگزادسیمال مثل 0x400 هیچ معنای خاصی برای انسان ندارند. اگر شش ماه بعد به این کد نگاه کنید، بدون داشتن دفترچه راهنمای هزار صفحه‌ای تراشه، محال است بفهمید این خطوط چه کاری انجام می‌دهند. همچنین احتمال خطای انسانی در محاسبه باینری این اعداد بسیار بالاست.

۲. راه حل مدرن؛ لایه انتزاعی سخت‌افزار یا HAL

شرکت STMicroelectronics برای حل این پیچیدگی، راهکاری استاندارد ارائه کرده است که به آن HAL یا Hardware Abstraction Layer می‌گویند.

فلسفه HAL پنهان کردن پیچیدگی‌های سخت‌افزاری پشت توابع استاندارد زبان C است. این لایه مانند یک واسط هوشمند عمل می‌کند. شما به عنوان برنامه‌نویس فقط قصد و نیت خود را بیان می‌کنید و کتابخانه HAL زحمت محاسبه بیت‌ها و آدرس‌ها را می‌کشد.

این دقیقاً مشابه مفهوم کلاس و متد در برنامه‌نویسی شیء‌گرا است. شما نیازی ندارید بدانید داخل متد چه می‌گذرد، فقط باید بدانید ورودی و خروجی آن چیست.

۳. مقایسه عملی؛ شفافیت در برابر ابهام

بیاید همان عملیات روشن کردن پین شماره ۵ از پورت A را این بار با زبان HAL بنویسیم:

```
1. // HAL Library - The Smart Way
2. HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
```

تفاوت چشمگیر است:

- **خوانایی:** کد مثل زبان انگلیسی خوانده می‌شود. "روی پین ۵ از پورت A مقدار SET را بنویس."
- **قابلیت حمل:** اگر بخواهید این کد را روی یک میکروکنترلر دیگر از خانواده STM32 اجرا کنید، نیازی به تغییر آدرس‌های حافظه نیست و توابع HAL در تمام مدل‌ها یکسان هستند.
- **عیب‌یابی:** احتمال خطا تقریباً صفر است چون شما با اعداد سروکار ندارید.

۴. ساختار درایور HAL؛ زیر پوست کد چه می‌گذرد؟

شاید برایتان سوال شود که این توابع کجا تعریف شده‌اند. وقتی با ابزار CubeMX پروژه می‌سازیم، تعدادی فایل با پسوند C به پروژه اضافه می‌شوند. برای مثال فایل `stm32f4xx_hal_gpio.c` حاوی تمام توابع مربوط به پورت‌های ورودی و خروجی است. وقتی شما تابع `HAL_GPIO_WritePin` را صدا می‌زنید، پردازنده به داخل این فایل می‌رود. در آنجا کدهایی نوشته شده است که همان عملیات بیتی و مقداردهی به رجیسترها را انجام می‌دهند. پس در واقع HAL جادو نمی‌کند، بلکه فقط کدهای سطح پایین و سخت را بسته‌بندی کرده و با یک برچسب تمیز به ما تحویل می‌دهد.

۵. نتیجه‌گیری؛ انتخاب مسیر مهندسی

در این دوره آموزشی ما مسیر HAL را انتخاب می‌کنیم. اگرچه دسترسی مستقیم به رجیسترها سرعت اجرای بالاتری دارد و حجم کد کمتری تولید می‌کند، اما در پروژه‌های مدرن و پیچیده، **زمان توسعه** و **قابلیت نگهداری کد** اهمیت بیشتری دارد. استفاده از HAL به ما اجازه می‌دهد تا به جای درگیر شدن با جزئیات بیتی، روی منطق اصلی برنامه و الگوریتم‌های هوشمند تمرکز کنیم. اکنون که زبان مشترک خود با سخت‌افزار را پیدا کردیم، آماده‌ایم تا در فصل بعد اولین پروژه عملی خود را آغاز کنیم.

فصل ۳: اولین ضربان؛ پروژه چشمک‌زن با طعم HAL

در دنیای برنامه‌نویسی کامپیوتر اولین برنامه همیشه چاپ کردن Hello World است. در دنیای میکروکنترلرها، سلام کردن به دنیا با روشن و خاموش کردن یک LED انجام می‌شود. ما قصد داریم یک LED متصل به پایه PA5 را با فاصله زمانی دقیق ۵۰۰ میلی‌ثانیه روشن و خاموش کنیم.

۱. خداحافظی با کدنویسی تنظیمات؛ جادوی CubeMX

در معماری‌های قدیمی برای اینکه یک پایه را خروجی کنید، باید بیت‌های رجیستر جهت‌نما را پیدا کرده و مقداری می‌کردید. اما در STM32 ما ابزاری به نام CubeMX داریم که بخشی از محیط برنامه‌نویسی شماست.

مراحل تنظیم سخت‌افزار به روش گرافیکی:

۱. نرم‌افزار STM32CubeIDE را باز کنید و یک پروژه جدید با انتخاب چیپ STM32F401RE بسازید. ۲. پس از چند لحظه تصویر گرافیکی چیپ را می‌بینید. ۳. پایه PA5 را روی تصویر پیدا کنید و روی آن کلیک چپ کنید. ۴. از لیستی که باز می‌شود گزینه **GPIO_Output** را انتخاب کنید. تمام شد. با همین چند کلیک ساده، شما هزاران خط کد مربوط به فعال‌سازی کلاک پورت A و تنظیمات رجیستر MODER را انجام دادید. رنگ پایه سبز می‌شود که نشان‌دهنده تنظیم شدن آن است.

۲. تنظیمات قلب تپنده؛ پیکربندی کلاک

یکی از مهم‌ترین بخش‌ها در میکروکنترلرهای ARM تنظیم سرعت کلاک است. برخلاف AVR که معمولاً روی ۱ مگاهرتز پیش‌فرض کار می‌کرد، اینجا ما قدرت مانور بسیار بالایی داریم. برای اینکه شبیه‌سازی ما در پروتئوس دقیق کار کند، باید سرعت را روی عدد مشخصی تنظیم کنیم.

۱. در منوی سمت چپ وارد بخش **System Core** شوید و گزینه **RCC** را انتخاب کنید. ۲. در قسمت **High Speed Clock** گزینه **Crystal/Ceramic Resonator** را انتخاب کنید. ۳. حالا

به تب **Clock Configuration** در بالای صفحه بروید. ۴۰ در کادر وسط صفحه که نوشته شده HCLK، عدد **84** را تایپ کرده و ایتر بزنید. نرم افزار به صورت خودکار تمام ضرب کننده ها و تقسیم کننده های داخلی (PLL) را محاسبه می کند تا سرعت پردازنده دقیقاً روی ۸۴ مگاهرتز تنظیم شود.

۳. تولید کد و ورود به محیط برنامه

حالا کلیدهای Ctrl+S را فشار دهید. نرم افزار از شما می پرسد که آیا کدها را تولید کند؟ بله را انتخاب کنید. فایل main.c باز می شود. اگر پایین بروید می بینید که توابع تنظیمات سخت افزار به صورت خودکار نوشته شده اند.

نکته حیاتی برای برنامه نویسان: در کدهای تولید شده، بخش هایی با کامنت **USER CODE BEGIN** و **USER CODE END** مشخص شده اند. همیشه کدهای خودتان را فقط در بین این خطوط بنویسید. اگر جای دیگری کد بنویسید، با هر بار تغییر تنظیمات گرافیکی، کدهای شما پاک خواهند شد.

۴. کدنویسی؛ فقط ۲ خط!

حالا به داخل تابع main و حلقه while(1) بروید. ما می خواهیم LED را خاموش و روشن کنیم. کتابخانه HAL دو تابع قدرتمند برای این کار به ما می دهد. تابع اول: **تغییر وضعیت پین** به جای اینکه بنویسیم "اگر صفر بود یک کن و اگر یک بود صفر کن"، از تابع Toggle استفاده می کنیم که وضعیت پین را معکوس می کند:

```
1. HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
```

این دستور به سادگی می گوید: در پورت A، پین شماره ۵ را معکوس کن.

تابع دوم: **ایجاد تاخیر دقیق** در معماری های ساده، تابع delay معمولاً پردازنده را در یک حلقه خالی نگه می داشت و دقت پایینی داشت. اما تابع HAL_Delay در اینجا از یک تایمر سخت افزاری دقیق به نام SysTick استفاده می کند که در پس زمینه سیستم می شمارد.

```
1. HAL_Delay(500);
```

این دستور دقیقاً ۵۰۰ میلی ثانیه صبر می کند.

کد نهایی در حلقه while:

```

1.  /* Infinite loop */
2.  /* USER CODE BEGIN WHILE */
3.  while (1)
4.  {
5.      /* USER CODE END WHILE */
6.
7.      /* USER CODE BEGIN 3 */
8.      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
9.      HAL_Delay(500);
10. }
11. /* USER CODE END 3 */

```

۵. شبیه‌سازی در پروتئوس

حالا که فایل Hex را ساختید (طبق آموزش فصل ۱)، وقت آن است که نتیجه را ببینیم.

۱. در پروتئوس میکروکنترلر STM32F401 را بیاورید.

۲. یک LED و یک مقاومت ۲۲۰ اهم به پایه PA5 وصل کنید.

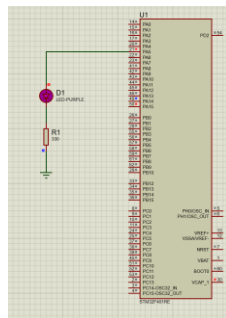
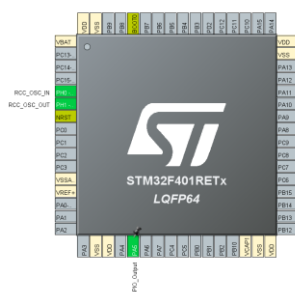
۳. روی میکروکنترلر دابل کلیک کنید .

۴. در بخش **Program File** آدرس فایل Hex خود را بدهید .

۵. نکته بسیار مهم: در بخش **Crystal Frequency** باید دقیقاً همان عددی را بنویسید که در تنظیمات

کلاک وارد کردید. یعنی **84MHz** اگر این عدد متفاوت باشد، سرعت چشمک زدن در شبیه‌سازی کند یا تند خواهد شد.

دکمه Play را بزنید. تبریک می‌گوییم! اولین پروژه ۳۲ بیتی شما با موفقیت اجرا شد. شما اکنون با موفقیت از سد تنظیمات پیچیده سخت‌افزاری عبور کردید و وارد دنیای HAL شدید.



فصل ۴: حس کردن محیط؛ ورودی‌ها و دکمه‌ها

در دنیای دیجیتال همه چیز به دو وضعیت صفر و یک ختم می‌شود. خواندن ورودی یعنی اینکه میکروکنترلر تشخیص دهد ولتاژ روی یک پایه خاص ۳.۳ ولت یا همان یک منطقی است یا اینکه صفر ولت یا همان زمین است. ما می‌خواهیم پروژه‌ای بسازیم که با فشردن دکمه LED روشن شود و با رها کردن آن خاموش گردد.

۱. خداحافظی با مقاومت‌های خارجی؛ مفهوم پول-آپ داخلی

شما به عنوان مهندس کامپیوتر می‌دانید که وضعیت نامشخص در منطق دیجیتال معنایی ندارد. اگر یک پایه ورودی را به هیچ کجا وصل نکنید و رها باشد اصطلاحاً شناور یا Floating نامیده می‌شود. در این حالت پایه مثل آنتن عمل کرده و نویز محیط را جذب می‌کند که باعث می‌شود میکروکنترلر به اشتباه گاهی یک و گاهی صفر بخواند.

برای رفع این مشکل در مدارهای قدیمی مجبور بودیم از یک مقاومت فیزیکی استفاده کنیم تا پایه را در حالت عادی به برق مثبت یا منفی متصل نگه داریم.

خبر خوب در: STM32 معماری ARM فکر همه جا را کرده است. داخل چیپ برای تک تک پایه‌ها مقاومت‌های داخلی تعبیه شده است که ما می‌توانیم با نرم‌افزار آن‌ها را فعال کنیم.

- **حالت Pull-up:** پایه از داخل به برق مثبت وصل می‌شود. در حالت عادی مقدار یک خوانده

می‌شود و ما دکمه را طوری می‌بندیم که با فشردن آن پایه به منفی وصل شود.

- **حالت Pull-down:** پایه از داخل به زمین وصل می‌شود. در حالت عادی مقدار صفر خوانده

می‌شود و دکمه باید برق مثبت را به پایه برساند.

ما در این پروژه از روش استاندارد صنعتی یعنی پول-آپ داخلی استفاده می‌کنیم.

۲. تنظیمات در CubeMX؛ ساده و سریع

پروژه قبلی را باز کنید یا پروژه جدیدی بسازید. ما فرض می‌کنیم LED همچنان به پایه PA5 متصل است. حالا می‌خواهیم دکمه‌ای به پایه PC13 وصل کنیم.

۱. روی پایه PC13 کلیک چپ کنید.

۲. گزینه **GPIO_Input** را انتخاب کنید.

۳. حالا نوبت تنظیم مقاومت داخلی است. از منوی سمت چپ وارد بخش **System Core** و سپس **GPIO** شوید.

۴. در وسط صفحه روی ردیف مربوط به PC13 کلیک کنید.

۵. در پنجره پایین صفحه بخشی به نام **GPIO Pull-up/Pull-down** وجود دارد. آن را روی حالت **Pull-up** قرار دهید.

با این کار به میکروکنترلر دستور دادید که مقاومت داخلی متصل به ۳.۳ ولت را برای این پایه فعال کند. حالا کد را تولید کنید.

۳. کدنویسی؛ تابع خواندن پین

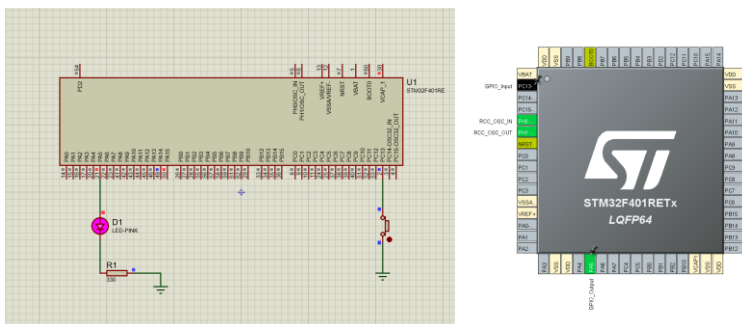
در کتابخانه HAL تابعی برای خواندن وضعیت پین‌ها وجود دارد که خروجی آن یا **GPIO_PIN_SET** به معنای یک و یا **GPIO_PIN_RESET** به معنای صفر است.

```
1. HAL_GPIO_ReadPin(GPIOx, GPIO_Pin);
```

منطق برنامه ما: چون از مقاومت پول-آپ استفاده کردیم منطق ما معکوس یا **Active Low** است.

- دکمه رها شده: ولتاژ ۳.۳ ولت است و ورودی یک خوانده می‌شود.
 - دکمه فشرده شده: پایه به زمین وصل می‌شود و ورودی صفر خوانده می‌شود.
- پس باید در کد بگوییم: "اگر پایه صفر بود یعنی دکمه فشرده شده است پس چراغ را روشن کن."
- وارد حلقه **while(1)** شوید و کد زیر را بنویسید:

```
1. while (1)
2. {
3.     /* USER CODE BEGIN 3 */
4.
5.     // Check if Button (PC13) is pressed
6.     // Remember: Pull-up means pressed = 0 (RESET)
7.     if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET)
8.     {
9.         // Turn LED ON
10.        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
11.    }
12.    else
13.    {
14.        // Turn LED OFF
15.        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
16.    }
17.
18.    // A small delay is good for stability
19.    HAL_Delay(10);
20.
21.    /* USER CODE END 3 */
22. }
```



۴. راهنمای اتصالات در پروتئوس (طبق تصویر)

برای اینکه شبیه‌سازی شما دقیقاً مانند تصویر ضمیمه شده کار کند، اتصالات را به این صورت انجام دهید:

۱. اتصال LED:

- پایه شماره ۲۱ میکروکنترلر که همان **PA5** است را به مثبت LED (بخش پهن‌تر) وصل کنید.
- پایه منفی LED (بخش خط‌دار) را به یک سر مقاومت ۳۳۰ اهم وصل کنید.
- سر دیگر مقاومت را به **Ground** متصل کنید.

۲. اتصال دکمه:

- یک **Button** از کتابخانه قطعات بیاورید.
- یک سر دکمه را به پایه شماره ۴ میکروکنترلر که همان **PC13** است وصل کنید.
- سر دیگر دکمه را مستقیم به **Ground** وصل کنید.

فصل ۵: نمایشگر کاراکتری؛ پورت کردن کتابخانه

تا اینجا فقط با LED و دکمه (۰ و ۱) کار کردیم. اما برای نمایش اعداد، دما یا پیام‌های متنی به یک نمایشگر نیاز داریم LCD. کاراکتری معمولاً مدل ۱۶ در ۲۰ ارزان‌ترین و محبوب‌ترین گزینه است. در این فصل یاد می‌گیریم چگونه پورتکل ارتباطی LCD را با استفاده از توابع HAL شبیه‌سازی کنیم و متن‌های پویا (مثل شمارنده) را نمایش دهیم.

۱. سخت‌افزار در پروتئوس؛ سیم‌کشی ۴ بیتی

ما برای صرفه‌جویی در پین‌ها، از مد ۴ بیتی استفاده می‌کنیم. یعنی به جای ۸ سیم دیتا، فقط ۴ سیم D4 تا D7 را وصل می‌کنیم. اتصالات در پروتئوس (طبق کد ما): یک قطعه LM016L (ال‌سی‌دی ۱۶ در ۲) و میکروکنترلر STM32F401RE بیاورید. اتصالات را به پورت B انجام دهید:

- پایه RS نمایشگر PB0 --->
- پایه E نمایشگر PB1 ---> (Enable)
- پایه RW نمایشگر GND ---> چون فقط می‌نویسیم)
- پایه D4 نمایشگر PB4 --->
- پایه D5 نمایشگر PB5 --->
- پایه D6 نمایشگر PB6 --->
- پایه D7 نمایشگر PB7 --->
- پایه‌های D0 تا D3 نمایشگر ---> آزاد (بدون اتصال)

۲. تنظیمات CubeMX

۱. پروژه جدیدی بسازید.

۲. پایه‌های **PB0, PB1, PB4, PB5, PB6, PB7** را پیدا کنید.

۳. روی تک‌تک آن‌ها کلیک کرده و گزینه **GPIO_Output** را انتخاب کنید.

۴. تنظیمات کلاک را (طبق فصل ۳) انجام دهید و کد را تولید کنید.

۳. کدنویسی؛ نوشتن درایور LCD در دل main.c

چون کتابخانه آماده نداریم، خودمان توابع لازم را می‌نویسیم. این کدها را باید در فایل **main.c** و در بخش‌های مشخص شده کپی کنید.

الف) افزودن کتابخانه استاندارد: برای استفاده از **sprintf** باید هدر فایل ورودی/خروجی استاندارد را اضافه کنیم. در بخش **/* USER CODE BEGIN Includes */**:

```
1. #include <stdio.h>
```

ب) تعریف توابع **LCD**: این توابع را قبل از **main** مثلاً در بخش

/* USER CODE BEGIN 0 */ کپی کنید. این در واقع همان کتابخانه **LCD** است که ما دستی

نوشتیم.

```
1. /* USER CODE BEGIN 0 */
2.
3. // --- LCD Pin Definitions ---
4. #define LCD_PORT GPIOB
5. #define RS_PIN GPIO_PIN_0
6. #define EN_PIN GPIO_PIN_1
7.
8. // --- Helper: Send 4-bits to LCD ---
9. void LCD_WriteNibble(uint8_t data, uint8_t rs_state) {
10.     // 1. Set RS (0 for Command, 1 for Data)
11.     HAL_GPIO_WritePin(LCD_PORT, RS_PIN, rs_state ? GPIO_PIN_SET :
GPIO_PIN_RESET);
12.
13.     // 2. Set Data Pins (D4..D7 connected to PB4..PB7)
14.     // We mask the bits to isolate D4-D7
15.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_4, (data >> 0) & 0x01);
16.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_5, (data >> 1) & 0x01);
17.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_6, (data >> 2) & 0x01);
18.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_7, (data >> 3) & 0x01);
```

```

19.
20.     // 3. Pulse Enable Pin (High -> Low) to latch data
21.     HAL_GPIO_WritePin(LCD_PORT, EN_PIN, GPIO_PIN_SET);
22.     HAL_Delay(1); // Wait > 450ns
23.     HAL_GPIO_WritePin(LCD_PORT, EN_PIN, GPIO_PIN_RESET);
24.     HAL_Delay(1); // Wait > 37us
25. }
26.
27. // --- Send Full Byte (Command or Data) ---
28. void LCD_Send(uint8_t value, uint8_t rs_state) {
29.     // Send Higher Nibble (Upper 4 bits)
30.     LCD_WriteNibble((value >> 4) & 0x0F, rs_state);
31.
32.     // Send Lower Nibble (Lower 4 bits)
33.     LCD_WriteNibble(value & 0x0F, rs_state);
34. }
35.
36. // --- User Function: Send Command ---
37. void LCD_Cmd(uint8_t cmd) {
38.     LCD_Send(cmd, 0); // RS = 0
39. }
40.
41. // --- User Function: Send Data (Char) ---
42. void LCD_Data(char data) {
43.     LCD_Send(data, 1); // RS = 1
44. }
45.
46. // --- User Function: Initialize LCD ---
47. void LCD_Init() {
48.     HAL_Delay(50); // Wait for power up
49.
50.     // Magic Sequence for 4-bit mode initialization
51.     LCD_Cmd(0x02);
52.     LCD_Cmd(0x28); // 2 lines, 5x7 matrix
53.     LCD_Cmd(0x0C); // Display ON, Cursor OFF
54.     LCD_Cmd(0x06); // Increment cursor
55.     LCD_Cmd(0x01); // Clear Display
56.     HAL_Delay(2);
57. }
58.
59. // --- User Function: Send String ---
60. void LCD_String(char *str) {
61.     while (*str) {
62.         LCD_Data(*str++);
63.     }
64. }
65.
66. // --- User Function: Set Cursor Position ---
67. void LCD_SetCursor(uint8_t row, uint8_t col) {
68.     // Row 0 starts at 0x80, Row 1 starts at 0xC0
69.     uint8_t address = (row == 0) ? 0x80 : 0xC0;
70.     address += col;
71.     LCD_Cmd(address);
72. }
73.

```

74. /* USER CODE END 0 */

ج) برنامه اصلی: (Main Loop) حالا وارد تابع main شوید. ما می‌خواهیم یک شمارنده بسازیم که عددش زیاد می‌شود.

```

1.  /* USER CODE BEGIN 2 */
2.
3.  // 1. Initialize LCD
4.  LCD_Init();
5.
6.  // 2. Print Static Text
7.  LCD_SetCursor(0, 0); // Row 0, Col 0
8.  LCD_String("Hello world!");
9.
10. int counter = 0;
11. char buffer[16]; // Buffer to hold formatted text
12.
13. /* USER CODE END 2 */
14.
15. while (1)
16. {
17.     /* USER CODE BEGIN 3 */
18.
19.     // 3. Format the counter into a string
20.     // sprintf is "String Print Format". It writes to 'buffer' instead of
screen.
21.     sprintf(buffer, "Count: %d  ", counter);
22.
23.     // 4. Move cursor to second line and print
24.     LCD_SetCursor(1, 0); // Row 1, Col 0
25.     LCD_String(buffer);
26.
27.     // 5. Logic
28.     counter++;
29.     HAL_Delay(500); // Update every half second
30.
31.     /* USER CODE END 3 */
32. }
```

۴. تحلیل فنی: تابع sprintf چیست؟

در زبان C ما نمی‌توانیم مستقیماً یک عدد int را به تابع LCD_String بدهیم، چون این تابع فقط آرایه‌ای از کاراکترها (متن) را می‌پذیرد. تابع sprintf که در کتابخانه stdio.h است دقیقاً کار printf را انجام می‌دهد، اما خروجی را به جای مانیتور، درون یک آرایه (Buffer) می‌نویسد.

- %d: جای‌گذاری عدد دهمی.
- buffer: آرایه‌ای که نتیجه نهایی مثلاً "Count: 15" در آن ذخیره می‌شود.

۵. تست در پروتئوس

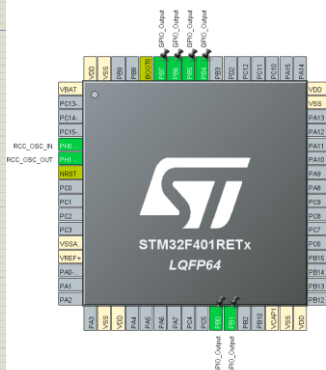
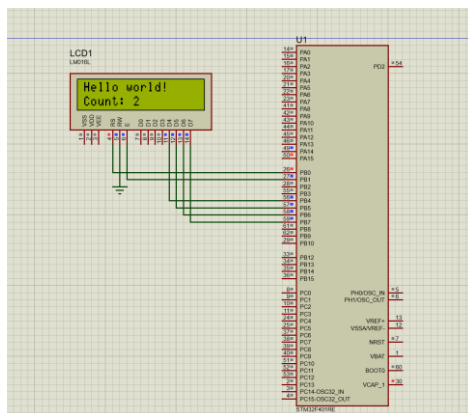
۱. فایل Hex را بسازید.

۲. در پروتئوس روی میکروکنترلر دابل کلیک کنید و فایل را انتخاب کنید.

۳. فرکانس کریستال را چک کنید. (84MHz)

۴. اجرا کنید.

باید ببینید که خط اول نوشته شده "Hello world!" و در خط دوم شمارنده هر نیم ثانیه زیاد می‌شود. اگر متن‌ها به هم ریخته بود، احتمالاً ترتیب اتصال پایه‌های D4 تا D7 را در پروتوس اشتباه بسته‌اید. دقت کنید که PB4 به D4 و PB7 ... به D7 متصل باشد.



فصل ۶: توقف ممنوع؛ جادوی وقفه‌ها

در فصل‌های قبل برای خواندن کلید مجبور بودیم داخل حلقه اصلی مدام وضعیت پایه را چک کنیم. به این روش **Polling** یا سرکشی مداوم می‌گویند. این کار دقیقاً مثل این است که منتظر مهمان باشید و به جای اینکه به کارهای خانه برسید پشت در بنشینید و مدام از چشمی در بیرون را نگاه کنید. اما روش هوشمندانه‌تر چیست؟ نصب زنگ در. شما با خیال راحت به کارهای خود می‌رسید و هر وقت مهمان زنگ زد کارتان را رها می‌کنید در را باز می‌کنید و دوباره برمی‌گردید. در دنیای ARM به این زنگ در وقفه یا **Interrupt** می‌گویند.

۱. سناریوی پروژه؛ دوستی خاله خرسه

برای اینکه تفاوت اجرای خطی و وقفه را درک کنیم این مدار را می‌بندیم:

- **وظیفه اصلی:** یک LED سبز به پایه **PA5** وصل است که باید دائماً با سرعت مشخصی چشمک بزند. این کار اصلی CPU است.
- **وظیفه اورژانسی:** یک LED قرمز به پایه **PA6** وصل است. این چراغ خاموش است اما هر بار که دکمه متصل به **PC13** فشرده شد باید بلافاصله تغییر وضعیت دهد. نکته مهم اینجاست که ما برای دکمه هیچ کدی در حلقه اصلی نمی‌نویسیم.

۲. تنظیمات در CubeMX؛ فعال‌سازی زنگ خطر

پروژه جدیدی بسازید و مراحل زیر را با دقت انجام دهید زیرا تنظیمات وقفه کمی متفاوت است.

الف. تنظیم خروجی‌ها

۱. روی پایه **PA5** کلیک کرده و **GPIO_Output** را انتخاب کنید.
۲. روی پایه **PA6** کلیک کرده و **GPIO_Output** را انتخاب کنید.

ب. تنظیم ورودی وقفه

۱. روی پایه **PC13** کلیک کنید.
۲. این بار به جای **Input** گزینه **GPIO_EXTI13** را انتخاب کنید. کلمه **EXTI** مخفف **External Interrupt** است یعنی وقفه‌ای که از بیرون تراشه می‌آید.

ج. تنظیمات **NVIC**؛ مغز متفکر وقفه‌ها در میکروکنترلرهای **ARM** واحدی به نام **NVIC** وجود دارد که مسئول مدیریت تمام وقفه‌هاست. ما باید به این واحد بگوییم که حواسش به خط شماره ۱۳ باشد.

۱. از منوی سمت چپ وارد بخش **System Core** و سپس **NVIC** شوید

۲. در لیست وسط صفحه گزینه‌ای به نام **EXTI line[15:10] interrupts** را پیدا کنید.

۳. تیک ستون **Enabled** را بزنید تا فعال شود.

د. تنظیمات دکمه

۱. وارد بخش **GPIO** شوید.

۲. روی ردیف **PC13** کلیک کنید.

۳. در تنظیمات پایین صفحه گزینه **GPIO Pull-up/Pull-down** را روی **Pull-up** بگذارید.

۴. گزینه **GPIO mode** را روی **External Interrupt Mode with Falling edge**

trigger قرار دهید. این یعنی وقتی ولتاژ از بالا به پایین آمد یا همان لحظه فشرده شدن دکمه وقفه رخ دهد.

حالا کد را تولید کنید.

۳. کدنویسی؛ تابع **Callback** کجاست؟

در کتابخانه **HAL** برخلاف **AVR** ما تابع وقفه را مستقیم نمی‌نویسیم. وقتی وقفه رخ می‌دهد کتابخانه **HAL** کارهای اولیه را انجام می‌دهد و سپس تابعی به نام **Callback** را صدا می‌زند. ما باید کدمان را درون این تابع بنویسیم.

این تابع در فایل **main.c** وجود ندارد و باید خودمان آن را اضافه کنیم. بهترین جا برای نوشتن این تابع در انتهای فایل **main.c** و در بخش **USER CODE BEGIN 4** است.

کد زیر را به انتهای فایل اضافه کنید:

```
1. /* USER CODE BEGIN 4 */
2.
3. // This function is called automatically when an interrupt occurs
4. void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
5. {
6.     // Check if the interrupt came from PC13
7.     if(GPIO_Pin == GPIO_PIN_13)
8.     {
9.         // Toggle the Red LED (PA6)
10.        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6);
11.    }
```

```
12. }  
13.  
14. /* USER CODE END 4 */
```

حالا وارد تابع `main` و حلقه `while` شوید و کد چشمک‌زن سبز را بنویسید:

```
1. while (1)  
2. {  
3.     /* USER CODE BEGIN 3 */  
4.  
5.     // Main Task: Blink Green LED slowly  
6.     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  
7.     HAL_Delay(500); // 500ms delay  
8.  
9.     // Notice: There is NO code here for the Button!  
10.  
11.    /* USER CODE END 3 */  
12. }
```

۴. تحلیل فنی؛ چه اتفاقی می‌افتد؟

بیایید صحنه آهسته را مرور کنیم:

۱. پردازنده در حلقه `while` مشغول است و دارد ۵۰۰ میلی‌ثانیه صبر می‌کند. در این لحظه CPU عملاً خواب است.

۲. شما دکمه را فشار می‌دهید.

۳. واحد NVIC سیگنال را می‌گیرد و می‌بیند که اولویت بالایی دارد.

۴. پردازنده بلافاصله عملیات صبر کردن را رها می‌کند و به تابع `HAL_GPIO_EXTI_Callback` می‌پرد.

۵. چراغ قرمز تغییر وضعیت می‌دهد. این کار شاید چند میکروثانیه طول بکشد.

۶. پردازنده دوباره به حلقه `while` برمی‌گردد و ادامه ۵۰۰ میلی‌ثانیه را صبر می‌کند. شما احساس می‌کنید دو کار همزمان انجام شده است.

۵. بستن مدار در پروتئوس

فایل Hex را بسازید و به پروتئوس بروید.

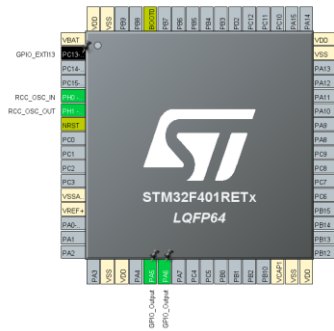
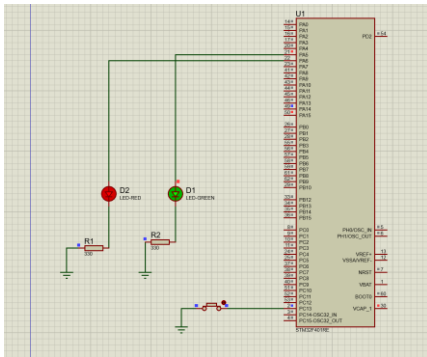
اتصالات :

۱. LED سبز: آند به پایه PA5 و کاتد با مقاومت به زمین.

۲. LED. قرمز: آند به پایه PA6 و کاتد با مقاومت به زمین.

۳. دکمه: یک سر به پایه PC13 و سر دیگر مستقیم به زمین.

تست: شبیه‌سازی را اجرا کنید. چراغ سبز به آرامی چشمک می‌زند. حالا دکمه را فشار دهید. می‌بینید که چراغ قرمز **آنی** و بدون هیچ تاخیری روشن و خاموش می‌شود حتی اگر چراغ سبز وسط زمان خاموشی خود باشد. این قدرت وقفه است که به سیستم شما سرعت واکنش بالا می‌دهد.



فصل ۷: مکالمه دوطرفه؛ ساخت کنسول فرمان با UART

تا این لحظه میکروکنترلر ما مانند یک جعبه در بسته عمل می‌کرد. اما در سیستم‌های پیشرفته نیاز داریم با سخت‌افزار صحبت کنیم. در این فصل یک **CLI** یا رابط خط فرمان می‌سازیم. سناریو این است که شما در کامپیوتر عدد **1** را تایپ می‌کنید و **LED** روشن می‌شود و سیستم به شما پاسخ می‌دهد. اگر **0** را بزنید خاموش می‌شود. ما تمام این کارها را بدون معطل کردن **CPU** و با استفاده از تکنیک وقفه انجام می‌دهیم.

۱. تعریف متغیرها؛ ظرفی برای داده‌ها

اولین چیزی که نیاز داریم یک متغیر است تا کاراکتری که از سمت کامپیوتر می‌آید را در آن ذخیره کنیم. در کدهای تولید شده به بخش متغیرهای خصوصی بروید و خط زیر را اضافه کنید.

در فایل **main.c** بخش **USER CODE BEGIN PV**:

```
1. /* USER CODE BEGIN PV */
2. uint8_t rx_byte; // Variable to store the single received character
3. /* USER CODE END PV */
```

۲. قلب تپنده پروژه؛ تابع Callback

این بخش مهم‌ترین قسمت کد شماست که **CubeMX** آن را تولید نمی‌کند و باید دستی بنویسید. وقتی کاراکتری از کامپیوتر به میکروکنترلر می‌رسد پردازنده کار جاری خود را رها می‌کند و به این تابع می‌پردازد. ما در اینجا بررسی می‌کنیم چه دستوری آمده و **LED** را کنترل می‌کنیم. این کد را قبل از تابع **main** در بخش **USER CODE BEGIN 0** قرار دهید:

```
1. /* USER CODE BEGIN 0 */
2.
3. /**
4.  * @brief Rx Transfer completed callback.
5.  * @param huart UART handle.
6.  * @retval None
7.  */
```

```

8. void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
9. {
10.    // Check if the interrupt comes from USART2
11.    if (huart->Instance == USART2)
12.    {
13.        // Check the received character
14.        if (rx_byte == '1')
15.        {
16.            // Turn Green LED ON (PA5)
17.            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
18.
19.            // Send Feedback to Terminal
20.            char msg[] = "COMMAND EXECUTED: LED ON\r\n";
21.            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
22.        }
23.        else if (rx_byte == '0')
24.        {
25.            // Turn Green LED OFF (PA5)
26.            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
27.
28.            // Send Feedback to Terminal
29.            char msg[] = "COMMAND EXECUTED: LED OFF\r\n";
30.            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
31.        }
32.        else
33.        {
34.            // Handle Invalid Command
35.            char msg[] = "ERROR: Unknown Command. Please send 1 or 0.\r\n";
36.            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
37.        }
38.
39.        // *** IMPORTANT: Re-enable the Interrupt for the next character ***
40.        HAL_UART_Receive_IT(&huart2, &rx_byte, 1);
41.    }
42. }
43.
44. /* USER CODE END 0 */

```

تحلیل فنی: نکته کلیدی خط آخر یعنی HAL_UART_Receive_IT است. در کتابخانه HAL

وقفه دریافت یکبار مصرف است. یعنی بعد از گرفتن یک کاراکتر غیرفعال می‌شود. ما باید قبل از خروج از تابع دوباره آن را شارژ کنیم تا برای دستور بعدی آماده باشد.

۳. شروع برنامه؛ تابع Main

حالا باید در شروع برنامه یک پیام خوش آمدگویی بفرستیم و سیستم شنود را فعال کنیم. وارد تابع `main` شوید و درست قبل از حلقه `while` کدهای زیر را بنویسید.

در بخش: USER CODE BEGIN 2:

```
1.  /* USER CODE BEGIN 2 */
2.
3.  // Send Welcome Message
4.  char ready_msg[] = "System Ready. Type '1' for ON, '0' for OFF...\r\n";
5.  HAL_UART_Transmit(&huart2, (uint8_t*)ready_msg, strlen(ready_msg), 100);
6.
7.  // Enable UART Receiver in Interrupt Mode (Listening for 1 byte)
8.  HAL_UART_Receive_IT(&huart2, &rx_byte, 1);
9.
10. /* USER CODE END 2 */
```

توجه کنید که حلقه `while` شما کاملاً خالی باقی می ماند. این یعنی CPU در حالت عادی بیکار است و تمام کارها به صورت رویدادمحور یا Event Driven در تابع Callback انجام می شود.

۴. شبیه سازی نهایی در پروتئوس

حالا که کد را کامل کردید فایل Hex را بسازید و در پروتئوس اجرا کنید.

تنظیمات حیاتی پروتئوس

۱. روی میکروکنترلر دابل کلیک کنید.

۲. مقدار **Forced HCLK** را دقیقاً روی **16000000** تنظیم کنید.

۳. اتصالات ترمینال مجازی را چک کنید:

- **RXD** ترمینال به PA2 میکروکنترلر متصل شود.
- **TXD** ترمینال به PA3 میکروکنترلر متصل شود.

۴. روی ترمینال دابل کلیک کنید و **Baud Rate** را روی **9600** بگذارید.

نحوه تست: شبیه سازی را اجرا کنید. روی پنجره سیاه ترمینال یک بار کلیک چپ کنید تا انتخاب شود.

حالا عدد ۱ و ۰ را روی کیبورد بزنید و نتیجه را ببینید.

۵. راهنمای تنظیم دقیق کلاک روی ۱۶ مگاهرتز

برای اینکه کد تولید شده دقیقاً با تنظیمات پروتئوس هماهنگ باشد و خطای تقسیم‌کننده کلاک نداشته باشیم، تنظیمات **CubeMX** را طبق مراحل زیر انجام دهید.

مراحل را به ترتیب در سریرگ **Clock Configuration** انجام دهید:

۱. انتخاب منبع کلاک در سمت چپ تصویر مطمئن شوید که باکس آبی رنگ **HSI RC 16 MHz** روشن است. این یعنی ما می‌خواهیم از نوسان‌ساز داخلی خود چپ استفاده کنیم.

۲. تغییر مسیر کلاک یا **System Clock Mux** در وسط تصویر یک سلکتور یا کلید بزرگ به نام **System Clock Mux** وجود دارد.

- وضعیت فعلی احتمالی: روی دایره پایین یعنی **PLLCLK** قرار دارد.

- تغییر: روی دایره بالا یعنی **HSI** کلیک کنید.

- دلیل: با این کار مسیر کلاک مستقیم می‌شود و از تمام پیچیدگی‌های **PLL** عبور می‌کند.

۳. تنظیم سرعت نهایی یا **HCLK** در سمت راست تصویر یک کادر آبی رنگ به نام **HCLK MHz** وجود دارد.

- تغییر: عدد داخل آن را پاک کنید و عدد **16** را بنویسید و **Enter** بزنید.

- نرم‌افزار به صورت خودکار تمام کادرها را به‌روزرسانی می‌کند.

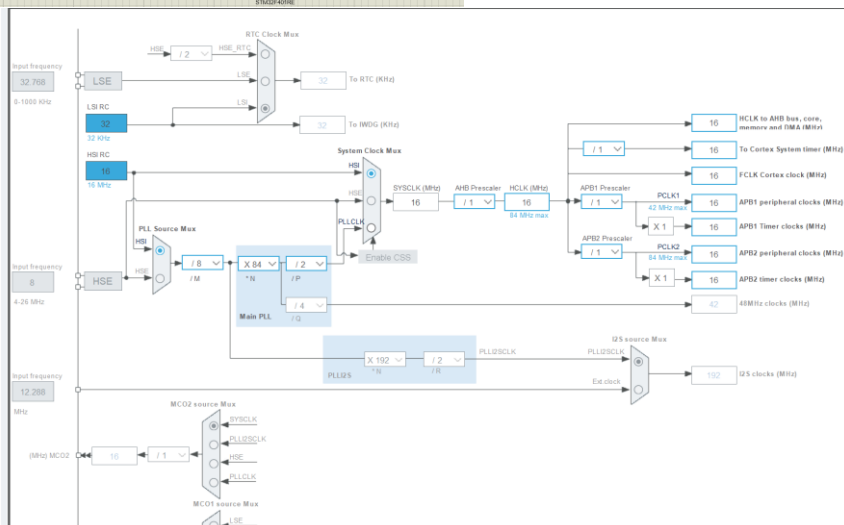
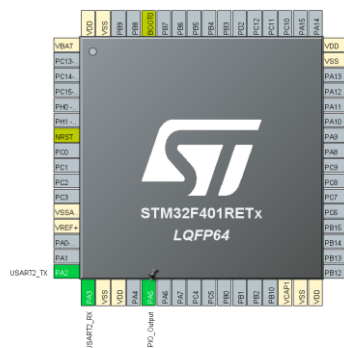
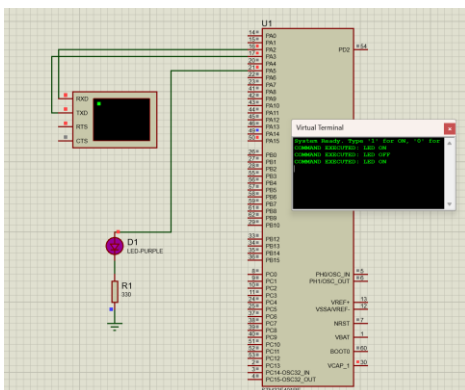
۴. تنظیم **APB1 Prescaler** حالا به کادر **APB1 Prescaler** نگاه کنید که در سمت راست و کمی پایین‌تر قرار دارد.

- وضعیت فعلی احتمالی: مقدار آن **2** است.

- تغییر: روی فلش کلیک کنید و مقدار آن را به **1** / یعنی تقسیم بر ۱ تغییر دهید.

چرا الان می‌توانیم روی ۱ بگذاریم؟ چون سرعت کل سیستم به ۱۶ مگاهرتز کاهش یافته است. با **APB1** توانایی تحمل ۴۲ مگاهرتز را دارد، پس ۱۶ مگاهرتز برایش هیچ فشاری ایجاد نمی‌کند و نیازی به نصف کردن سرعت نیست.

نکته: تنظیمات کلاک رو دقیقاً همینجوری انجام بدید وگرنه در خروجی ترمینال کاراکترهای اشتباهی بهتون نشون میده



فصل ۸: مدیریت زمان؛ راه اندازی تایمرهای مستقل

هدف ما در این پروژه ساخت یک چراغ راهنمایی پیشرفته است که دو LED با سرعت‌های متفاوت چشمک می‌زنند.

۱. LED سبز: هر ۵۰۰ میلی‌ثانیه یک بار تغییر وضعیت می‌دهد.

۲. LED قرمز: هر ۱۰۰ میلی‌ثانیه یک بار تغییر وضعیت می‌دهد (خیلی سریع).

نکته مهم این است که سرعت یکی نباید روی دیگری تأثیر بگذارد.

۱. مفهوم ریاضی تایمرها

تایمر در STM32 مثل یک ساعت شنی است که وقتی خالی شد یک زنگ (وقفه) می‌زند. ما باید دو پارامتر مهم را تنظیم کنیم:

۱. **Prescaler**: یا تقسیم‌کننده: سرعت کلاک ورودی را کم می‌کند تا شمارش قابل کنترل باشد.

۲. **Period**: یا همان **ARR** عددی است که تایمر تا آن می‌شمارد و سپس سرریز می‌کند.

فرمول ساده برای کلاک ۱۶ مگاهرتز: اگر بخواهیم تایمر ما هر ۱ میلی‌ثانیه یک قدم جلو برود باید کلاک ۱۶ میلیون را بر ۱۶۰۰۰ تقسیم کنیم. پس Prescaler ما همیشه 15999 خواهد بود. (چون شمارش از صفر شروع می‌شود یکی کم می‌کنیم).

حالا برای زمان‌های مختلف فقط عدد Period را تغییر می‌دهیم: برای ۵۰۰ میلی‌ثانیه عدد Period می‌شود 499 برای ۱۰۰ میلی‌ثانیه عدد Period می‌شود 99

۲. تنظیمات در CubeMX

پروژه قبلی را باز کنید یا پروژه جدید بسازید. تنظیمات کلاک را دقیقاً مثل فصل قبل روی 16MHz HSI بگذارید.

الف. تنظیم تایمر ۲ برای LED سبز

۱. وارد بخش **Timers** شوید و روی **TIM2** کلیک کنید.
۲. گزینه **Clock Source** را روی **Internal Clock** بگذارید.
۳. در پایین صفحه و بخش **Parameter Settings** اعداد زیر را وارد کنید:
 - **Prescaler: 15999**
 - **Counter Period: 499**
۴. وارد سربرگ **NVIC Settings** شوید و تیک **TIM2 global interrupt** را بزنید.

- ب. تنظیم تایمر ۳ برای LED قرمز
۱. در منوی **Timers** روی **TIM3** کلیک کنید.
 ۲. گزینه **Clock Source** را روی **Internal Clock** بگذارید.
 ۳. اعداد زیر را تنظیم کنید:
 - **Prescaler: 15999**
 - **Counter Period: 99**
 ۴. در سربرگ **NVIC Settings** تیک **TIM3 global interrupt** را بزنید.

- ج. تنظیم LED ها
۱. پایه **PA5** را خروجی کنید و نام آن را **LED_GREEN** بگذارید.
 ۲. پایه **PA0** را خروجی کنید و نام آن را **LED_RED** بگذارید.
- کد را تولید کنید.

۳. کدنویسی؛ مدیریت زمان

ما دیگر نیازی به نوشتن کد در حلقه **while** نداریم. همه چیز در تابع **Callback** انجام می شود.

الف. استارت زدن تایمرها تایمرها به صورت پیش فرض خاموش هستند. باید در شروع برنامه آن ها را روشن کنیم. وارد تابع **main** شوید و قبل از حلقه **while** کدهای زیر را بنویسید:

```
1. /* USER CODE BEGIN 2 */
2.
3. // Start Timer 2 in Interrupt mode (500ms)
4. HAL_TIM_Base_Start_IT(&tim2);
5.
6. // Start Timer 3 in Interrupt mode (100ms)
7. HAL_TIM_Base_Start_IT(&tim3);
```

```
8.
9. /* USER CODE END 2 */
```

ب. نوشتن تابع **Callback** مشترک وقتی هر کدام از تایمرها سرریز کنند میکروکنترلر به یک تابع واحد به نام **HAL_TIM_PeriodElapsedCallback** می‌رود. ما باید در این تابع چک کنیم که کدام تایمر زنگ زده است.

این کد را در بخش **USER CODE BEGIN 0** کپی کنید:

```
1. /* USER CODE BEGIN 0 */
2.
3. /**
4.  * @brief Period Elapsed Callback in non-blocking mode
5.  * @param htim: TIM handle
6.  * @retval None
7.  */
8. void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
9. {
10.    // Check if the interrupt comes from TIM2 (Green LED)
11.    if (htim->Instance == TIM2)
12.    {
13.        // Toggle Green LED every 500ms
14.        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
15.    }
16.
17.    // Check if the interrupt comes from TIM3 (Red LED)
18.    if (htim->Instance == TIM3)
19.    {
20.        // Toggle Red LED every 100ms
21.        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
22.    }
23. }
24.
25. /* USER CODE END 0 */
```

۴. شبیه‌سازی در پروتئوس

مدار را ببندید تا نتیجه کار موازی را ببینید.

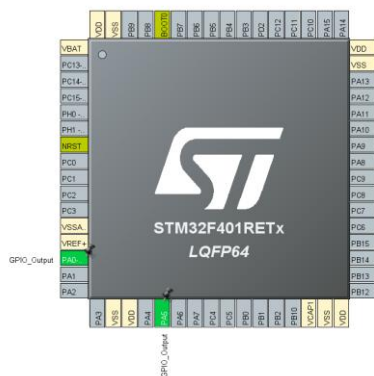
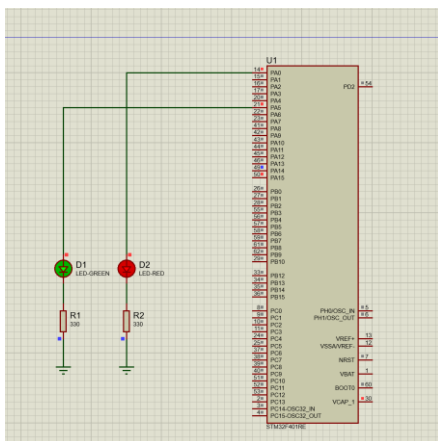
۱. فایل **Hex** را در میکروکنترلر بارگذاری کنید.

۲. فرکانس میکروکنترلر **Forced HCLK** حتماً روی **16000000** باشد.

۳. یک LED سبز به پایه **PA5** وصل کنید.

۴. یک LED قرمز به پایه **PA0** وصل کنید. ۵. دکمه **Play** را بزنید.

تحلیل نتیجه: شما خواهید دید که LED قرمز با سرعت زیاد چشمک می‌زند در حالی که LED سبز با آرامش کار خودش را می‌کند. هیچکدام منتظر دیگری نمی‌ماند. اگر از HAL_Delay استفاده می‌کردیم پردازنده مجبور بود ۵۰۰ میلی‌ثانیه صبر کند و در این مدت نمی‌توانست به LED قرمز رسیدگی کند اما با تایمرها این مشکل برای همیشه حل شد.



فصل ۹: هوشمندسازی دما؛ ساخت ترموستات اتوماتیک

تا این مرحله ما تنها با سیگنال‌های دیجیتال یعنی صفر و یک کار کردیم. اما دنیای واقعی ماهیت آنالوگ و پیوسته دارد. دما، نور و صدا همگی مقادیر متغیر هستند. در این فصل یاد می‌گیریم چگونه ولتاژ متغیر یک سنسور دما را بخوانیم و بر اساس آن تصمیم‌گیری کنیم. هدف ما ساخت یک ترموستات هوشمند است. ما از سنسور LM35 استفاده می‌کنیم که ولتاژ خروجی آن با دما تغییر می‌کند. سپس با واحد مبدل آنالوگ به دیجیتال یا همان ADC در میکروکنترلر این ولتاژ را به عدد تبدیل کرده و دما را محاسبه می‌کنیم.

۱. سناریوی پروژه

ما یک سیستم کنترل دما می‌سازیم که رفتار زیر را دارد:

- دمای کمتر از ۲۰ درجه: هوا سرد است پس هیتر یا همان LED قرمز روشن می‌شود.
- دمای بیشتر از ۳۰ درجه: هوا گرم است پس کولر یا همان LED آبی روشن می‌شود.
- دمای بین ۲۰ تا ۳۰ درجه: وضعیت نرمال است و هر دو خاموش هستند.
- دما و وضعیت سیستم باید به صورت لحظه‌ای روی نمایشگر LCD نمایش داده شود.

۲. سخت‌افزار و اتصالات در پروتئوس

برای بستن مدار در نرم‌افزار پروتئوس قطعات زیر را طبق دستورالعمل سیم‌کشی کنید:

الف. بخش سنسور و خروجی‌ها

۱. قطعه LM35 را به صفحه اضافه کنید. پایه خروجی سیگنال آن را به پایه PA0 میکروکنترلر وصل کنید.

۲. یک LED قرمز به پایه PA1 متصل کنید.

۳. یک LED آبی به پایه **PA2** متصل کنید. نکته: فراموش نکنید که سمت منفی **LED** ها را با یک مقاومت مناسب به زمین متصل کنید.
- ب. بخش نمایشگر **LCD** اتصالات نمایشگر کاراکتری ۱۶ در ۲ را دقیقاً به این صورت انجام دهید:
۱. پایه **RS** نمایشگر به پایه **PB0** میکروکنترلر وصل شود.
 ۲. پایه **EN** یا **Enable** نمایشگر به پایه **PB1** میکروکنترلر وصل شود.
 ۳. پایه **RW** نمایشگر را مستقیم به زمین یا **Ground** وصل کنید.
 ۴. پایه‌های دیتای **D4** تا **D7** نمایشگر را به ترتیب به پایه‌های **PB4** تا **PB7** میکروکنترلر وصل کنید.

۳. تنظیمات نرم‌افزاری در CubeMX

پروژه جدیدی بسازید یا پروژه قبلی را باز کنید و تنظیمات زیر را گام‌به‌گام انجام دهید:

گام اول: پیکربندی ADC

۱. از منوی سمت چپ وارد بخش **Analog** شوید و گزینه **ADC1** را انتخاب کنید.
۲. در کادر وسط تیک گزینه **IN0** را فعال کنید. این کار کانال صفر متصل به پایه **PA0** را روشن می‌کند.
۳. تنظیمات پیش‌فرض در پایین صفحه مناسب است و تغییری ندهید.

گام دوم: پیکربندی پایه‌های خروجی

۱. روی پایه **PA1** کلیک کنید و گزینه **GPIO_Output** را انتخاب نمایید.
۲. روی پایه **PA2** کلیک کنید و گزینه **GPIO_Output** را انتخاب نمایید.
۳. برای راه‌اندازی **LCD** باید پایه‌های پورت **B** را تنظیم کنیم. روی پایه‌های **PB0** و **PB1** و **PB4** و **PB5** و **PB6** و **PB7** تک‌تک کلیک کنید و همه را روی حالت **GPIO_Output** قرار دهید.

گام سوم: تنظیم کلاک ۱

وارد سربرگ **Clock Configuration** شوید.

۲. منبع کلاک **HSI** را انتخاب کنید.

۳. در کادر HCLK عدد 16 را وارد کرده و ایتر بزیند تا سرعت روی ۱۶ مگاهرتز تنظیم شود.

گام چهارم: تولید کد روی دکمه **Generate Code** کلیک کنید تا وارد محیط کدنویسی شویم.

۴. کدنویسی پروژه

در فایل **main.c** کدهای زیر را در بخش‌های مشخص‌شده کپی کنید. ما درایور LCD را همین‌جا می‌نویسیم تا نیازی به فایل خارجی نباشد.

بخش اول: فراخوانی کتابخانه‌های ضروری برای تبدیل اعداد به متن جهت نمایش روی LCD نیاز به کتابخانه استاندارد ورودی و خروجی داریم.

```
1. /* USER CODE BEGIN Includes */
2. #include <stdio.h>
3. /* USER CODE END Includes */
```

بخش دوم: تعریف توابع و درایور LCD این کدها را قبل از تابع **main** قرار دهید. این بخش وظیفه ارتباط با نمایشگر را بر عهده دارد.

```
1. /* USER CODE BEGIN 0 */
2.
3. // --- LCD Driver Definitions ---
4. #define LCD_PORT GPIOB
5. #define RS_PIN GPIO_PIN_0
6. #define EN_PIN GPIO_PIN_1
7.
8. // Helper function: Send 4-bits to LCD
9. void LCD_WriteNibble(uint8_t data, uint8_t rs_state) {
10.     // Set RS pin: 0 for Command, 1 for Data
11.     HAL_GPIO_WritePin(LCD_PORT, RS_PIN, rs_state ? GPIO_PIN_SET :
GPIO_PIN_RESET);
12.
13.     // Set Data pins (D4 to D7 connected to PB4 to PB7)
14.     // We utilize bitwise operations to set pins based on data bits
15.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_4, (data >> 0) & 0x01);
16.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_5, (data >> 1) & 0x01);
17.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_6, (data >> 2) & 0x01);
18.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_7, (data >> 3) & 0x01);
19.
20.     // Pulse Enable pin to latch data
21.     HAL_GPIO_WritePin(LCD_PORT, EN_PIN, GPIO_PIN_SET);
```



```
22.     HAL_Delay(1);
23.     HAL_GPIO_WritePin(LCD_PORT, EN_PIN, GPIO_PIN_RESET);
24.     HAL_Delay(1);
25. }
26.
27. // Function to send a full byte (Command or Data)
28. void LCD_Send(uint8_t value, uint8_t rs_state) {
29.     // Send Higher Nibble first
30.     LCD_WriteNibble((value >> 4) & 0x0F, rs_state);
31.     // Send Lower Nibble second
32.     LCD_WriteNibble(value & 0x0F, rs_state);
33. }
34.
35. // Command function for user
36. void LCD_Cmd(uint8_t cmd) {
37.     LCD_Send(cmd, 0); // RS = 0 for command
38. }
39.
40. // Data function for user to print single char
41. void LCD_Data(char data) {
42.     LCD_Send(data, 1); // RS = 1 for data
43. }
44.
45. // Initialization function
46. void LCD_Init() {
47.     HAL_Delay(50); // Wait for LCD power up
48.     // Standard initialization sequence for 4-bit mode
49.     LCD_Cmd(0x02);
50.     LCD_Cmd(0x28); // 2 lines, 5x7 matrix
51.     LCD_Cmd(0x0C); // Display ON, Cursor OFF
52.     LCD_Cmd(0x06); // Auto increment cursor
53.     LCD_Cmd(0x01); // Clear Display
54.     HAL_Delay(2);
55. }
56.
57. // Function to print a string
58. void LCD_String(char *str) {
59.     while (*str) {
60.         LCD_Data(*str++);
61.     }
62. }
63.
64. // Function to set cursor position
65. void LCD_SetCursor(uint8_t row, uint8_t col) {
66.     // Row 0 starts at 0x80, Row 1 starts at 0xC0
67.     uint8_t address = (row == 0) ? 0x80 : 0xC0;
68.     address += col;
69.     LCD_Cmd(address);
70. }
71.
72. /* USER CODE END 0 */
73.
```

بخش سوم: تعریف متغیرها ما به متغیرهایی برای ذخیره مقدار خام ADC و دمای محاسبه شده نیاز داریم.

```
1. /* USER CODE BEGIN PV */
2. uint32_t adc_raw_value = 0; // Variable to store raw ADC reading (0-4095)
3. int temperature_c = 0;      // Variable to store calculated temperature
   in Celsius
4. char display_buffer[16];    // Buffer array for LCD text formatting
5. /* USER CODE END PV */
```

بخش چهارم: بدنه اصلی برنامه این کدها را درون تابع main و حلقه while بنویسید.

```
1. /* USER CODE BEGIN 2 */
2.
3. // Initialize LCD before the loop
4. LCD_Init();
5.
6. // Show a welcome message briefly
7. LCD_SetCursor(0, 0);
8. LCD_String("Thermostat Sys");
9. LCD_SetCursor(1, 0);
10. LCD_String("Starting...");
11. HAL_Delay(1500);
12. LCD_Cmd(0x01); // Clear screen
13.
14. /* USER CODE END 2 */
15.
16. /* Infinite loop */
17. /* USER CODE BEGIN WHILE */
18. while (1)
19. {
20.     // --- Step 1: Read Sensor Data ---
21.
22.     // Start ADC Conversion
23.     HAL_ADC_Start(&hadc1);
24.
25.     // Wait for conversion to complete
26.     if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK)
27.     {
28.         // Get the converted value
29.         adc_raw_value = HAL_ADC_GetValue(&hadc1);
30.
31.         // Calculate Temperature
32.         // Formula: Temp = (ADC_Value * 330) / 4095
33.         // Based on 3.3V reference and 10mV/C sensor scale
34.         temperature_c = (adc_raw_value * 330) / 4095;
35.     }
36.
37.     // --- Step 2: Update Display ---
```

```

38.
39. // Display Temperature on Line 1
40. LCD_SetCursor(0, 0);
41. sprintf(display_buffer, "Temp: %d C ", temperature_c);
42. LCD_String(display_buffer);
43.
44. // --- Step 3: Control Logic ---
45.
46. LCD_SetCursor(1, 0); // Move to second line
47.
48. if (temperature_c < 20)
49. {
50.     // Condition: Cold -> Turn Heater (Red LED) ON
51.     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET); // Red
52.     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_RESET); // Blue
53.     LCD_String("Status: HEATING");
54. }
55. else if (temperature_c > 30)
56. {
57.     // Condition: Hot -> Turn Cooler (Blue LED) ON
58.     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET); // Red
59.     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_SET); // Blue
60.     LCD_String("Status: COOLING");
61. }
62. else
63. {
64.     // Condition: Normal -> All OFF
65.     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET); // Red
66.     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_RESET); // Blue
67.     LCD_String("Status: NORMAL ");
68. }
69.
70. // Small delay before next update
71. HAL_Delay(500);
72.
73. /* USER CODE END WHILE */
74. }
75.

```

۵. تست نهایی

فایل Hex ساخته شده را روی میکروکنترلر در پروتئوس بارگذاری کنید.

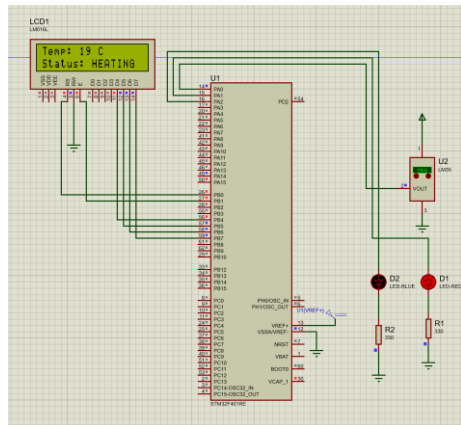
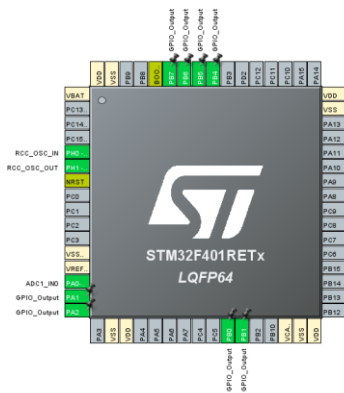
۱. برنامه را اجرا کنید.

۲. روی سنسور LM35 فلش‌های کوچکی وجود دارد که با کلیک روی آن‌ها می‌توانید دما را تغییر دهید.

۳. دما را به زیر ۲۰ ببرید و روشن شدن LED قرمز را مشاهده کنید.

۴. دما را به بالای ۳۰ ببرید و روشن شدن LED آبی را مشاهده کنید.

۵. وضعیت لحظه‌ای سیستم روی LCD نمایش داده می‌شود.



فصل ۱۰: ورودی‌های ماتریسی؛ اتصال ماشین حساب به میکروکنترلر

در این فصل ما یاد می‌گیریم که چگونه یک صفحه کلید ماتریسی را راه‌اندازی کنیم. مداری که شما بسته‌اید شامل یک کیپد ۴ در ۴ مدل ماشین حسابی است. ما از تکنیک "اسکن سطری و ستونی" استفاده می‌کنیم تا بفهمیم کدام کلید فشرده شده است.

۱. سخت‌افزار و اتصالات

مدار شما در پروتئوس صحیح است و نیازی به تغییر سیم‌کشی ندارید. اتصالات فعلی به شرح زیر است:

الف. بخش Keypad

- سطرها: (Rows A, B, C, D) به پایه‌های PA0, PA1, PA2, PA3 متصل هستند.
- ستون‌ها: (Cols 1, 2, 3, 4) به پایه‌های PA4, PA5, PA6, PA7 متصل هستند.

ب. بخش نمایشگر LCD

- پایه RS به PB0
- پایه EN به PB1
- پایه‌های دیتا D4 تا D7 به PB4 تا PB7

۲. تنظیمات نرم‌افزاری در CubeMX

برای اینکه کد درست کار کند باید تنظیمات پول‌آپ (Pull-up) را حتماً چک کنید.

گام اول: پیکربندی سطرها (خروجی) ۱. روی پایه‌های PA0, PA1, PA2, PA3 کلیک کنید ۲.

همه را روی حالت **GPIO_Output** قرار دهید.

گام دوم: پیکربندی ستون‌ها (ورودی با پول‌آپ)

۱. روی پایه‌های PA4, PA5, PA6, PA7 کلیک کنید.

۲. همه را روی حالت **GPIO_Input** قرار دهید.

۳. بسیار مهم: از منوی سمت چپ به بخش **GPIO** بروید. روی ردیف‌های مربوط به PA4 تا PA7 کلیک کنید و در پایین صفحه گزینه **Pull-up/Pull-down** را روی حالت **Pull-up** تنظیم کنید. بدون این کار، کلیدها کار نمی‌کنند.

گام سوم: تنظیم کلاک

۱. در سربرگ **Clock Configuration** سرعت **HCLK** را روی **16** مگاهرتز تنظیم کنید. دکمه **Generate Code** را بزنید.

۳. کدنویسی پروژه

کد زیر را کامل در فایل **main.c** جایگزین کنید یا در بخش‌های مربوطه قرار دهید. جدول کاراکترها (Keymap) دقیقاً طبق عکس شما تنظیم شده است.

بخش اول: هدر فایل‌ها

```
1. /* USER CODE BEGIN Includes */
2. #include <stdio.h>
3. /* USER CODE END Includes */
```

بخش دوم: درایور **LCD** و **Keypad** این کدها را قبل از تابع **main** در بخش **USER CODE BEGIN 0** کپی کنید.

```
1. /* USER CODE BEGIN 0 */
2.
3. // --- LCD Driver Configuration ---
4. #define LCD_PORT GPIOB
5. #define RS_PIN GPIO_PIN_0
6. #define EN_PIN GPIO_PIN_1
7.
8. // Function to send data nibble to LCD
9. void LCD_WriteNibble(uint8_t data, uint8_t rs_state) {
10.     HAL_GPIO_WritePin(LCD_PORT, RS_PIN, rs_state ? GPIO_PIN_SET :
GPIO_PIN_RESET);
11.
12.     // Set D4-D7 pins based on the data bits
13.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_4, (data >> 0) & 0x01);
14.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_5, (data >> 1) & 0x01);
15.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_6, (data >> 2) & 0x01);
```

```

16.     HAL_GPIO_WritePin(LCD_PORT, GPIO_PIN_7, (data >> 3) & 0x01);
17.
18.     // Toggle Enable pin to latch data
19.     HAL_GPIO_WritePin(LCD_PORT, EN_PIN, GPIO_PIN_SET);
20.     HAL_Delay(1);
21.     HAL_GPIO_WritePin(LCD_PORT, EN_PIN, GPIO_PIN_RESET);
22.     HAL_Delay(1);
23. }
24.
25. void LCD_Send(uint8_t value, uint8_t rs_state) {
26.     LCD_WriteNibble((value >> 4) & 0x0F, rs_state);
27.     LCD_WriteNibble(value & 0x0F, rs_state);
28. }
29.
30. void LCD_Cmd(uint8_t cmd) { LCD_Send(cmd, 0); }
31. void LCD_Data(char data) { LCD_Send(data, 1); }
32.
33. void LCD_Init() {
34.     HAL_Delay(50);
35.     LCD_Cmd(0x02); // Initialize 4-bit mode
36.     LCD_Cmd(0x28); // 2 lines, 5x7 matrix
37.     LCD_Cmd(0x0C); // Display ON, Cursor OFF
38.     LCD_Cmd(0x06); // Increment cursor
39.     LCD_Cmd(0x01); // Clear display
40.     HAL_Delay(2);
41. }
42.
43. void LCD_String(char *str) {
44.     while (*str) LCD_Data(*str++);
45. }
46.
47. void LCD_SetCursor(uint8_t row, uint8_t col) {
48.     uint8_t address = (row == 0) ? 0x80 : 0xC0;
49.     address += col;
50.     LCD_Cmd(address);
51. }
52.
53. // --- Calculator Keypad Mapping ---
54. // Based on the image provided:
55. // Row A: 7, 8, 9, /
56. // Row B: 4, 5, 6, x
57. // Row C: 1, 2, 3, -
58. // Row D: C, 0, =, +
59.
60. char keymap[4][4] = {
61.     {'7', '8', '9', '/'}, // Row 0 (PA0)
62.     {'4', '5', '6', 'x'}, // Row 1 (PA1)
63.     {'1', '2', '3', '-'}, // Row 2 (PA2)
64.     {'C', '0', '=', '+'}  // Row 3 (PA3)
65. };
66.
67. // Function to scan keypad

```

```

68. char Keypad_Read(void) {
69.     // Iterate through all 4 Rows
70.     for (int row = 0; row < 4; row++) {
71.
72.         // 1. Set all Rows to HIGH state
73.         HAL_GPIO_WritePin(GPIOA,
GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3, GPIO_PIN_SET);
74.
75.         // 2. Set ONLY the current Row to LOW state
76.         // We use bit shifting to select PA0, PA1, PA2, or PA3
77.         HAL_GPIO_WritePin(GPIOA, (1 << row), GPIO_PIN_RESET);
78.
79.         // Short delay for signal stability
80.         HAL_Delay(2);
81.
82.         // 3. Check all Columns (Inputs are Pull-Up, so Low means
Pressed)
83.         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_4) == GPIO_PIN_RESET)
return keymap[row][0];
84.         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) == GPIO_PIN_RESET)
return keymap[row][1];
85.         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_6) == GPIO_PIN_RESET)
return keymap[row][2];
86.         if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7) == GPIO_PIN_RESET)
return keymap[row][3];
87.     }
88.
89.     // Return 0 if no key is pressed
90.     return 0;
91. }
92.
93. /* USER CODE END 0 */

```

بخش سوم: متغیرها

```

1. /* USER CODE BEGIN PV */
2. char current_key = 0;    // Store the pressed key char
3. char display_buf[16];    // Buffer for LCD message
4. /* USER CODE END PV */

```

بخش چهارم: برنامه اصلی (Main Loop) این کدها را در حلقه while(1) قرار دهید.

```

1. /* USER CODE BEGIN 2 */
2. LCD_Init();
3.
4. LCD_SetCursor(0, 0);
5. LCD_String("Calculator Pad");
6. LCD_SetCursor(1, 0);
7. LCD_String("Press a Key...");

```



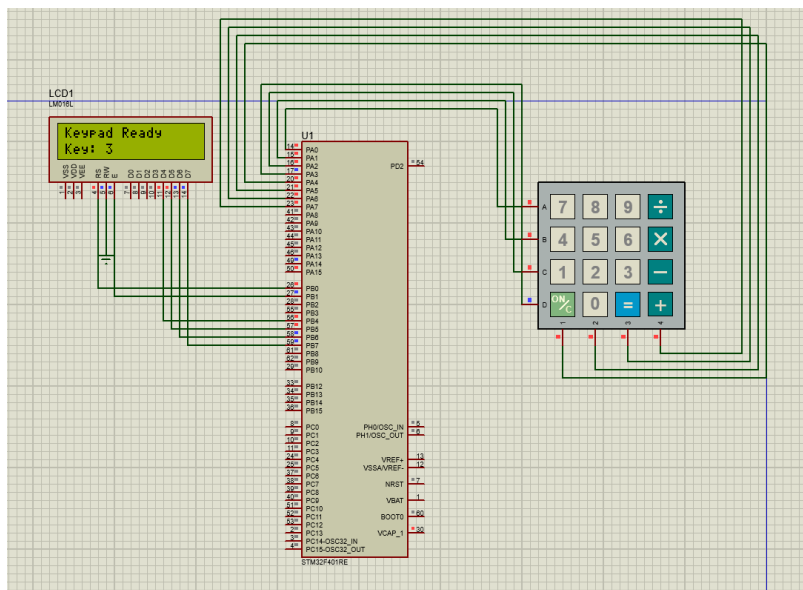
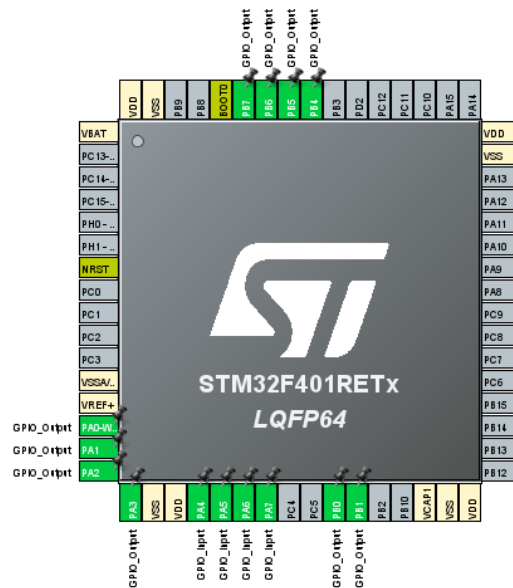
```

8.  /* USER CODE END 2 */
9.
10. /* Infinite loop */
11. /* USER CODE BEGIN WHILE */
12. while (1)
13. {
14.     // 1. Scan the keypad
15.     current_key = Keypad_Read();
16.
17.     // 2. If a key is pressed
18.     if (current_key != 0)
19.     {
20.         // Update LCD
21.         LCD_SetCursor(1, 0);
22.         LCD_String("                "); // Clear line
23.
24.         LCD_SetCursor(1, 0);
25.         sprintf(display_buf, "Pressed: %c", current_key);
26.         LCD_String(display_buf);
27.
28.         // 3. Debouncing and Locking
29.         // Wait here until the user releases the key
30.         while (Keypad_Read() != 0) {
31.             HAL_Delay(10);
32.         }
33.     }
34.
35.     HAL_Delay(50); // Scan rate
36.
37. /* USER CODE END WHILE */
38. }

```

۴. تست نهایی

۱. کد را کامپایل کنید و فایل Hex را بسازید.
 ۲. در پروتئوس روی دکمه 7 کلیک کنید -> روی LCD باید بنویسد **Pressed: 7**.
 ۳. روی دکمه **ON/C** کلیک کنید -> باید بنویسد **Pressed: C**.
 ۴. روی دکمه **ضرب (X)** کلیک کنید -> باید بنویسد **Pressed: x**.
- اگر روی ال‌سی‌دی چیزی ندیدید یا دکمه‌ها کار نکردند، اولین چیزی که باید چک کنید تنظیمات **Pull-up** پایه‌های PA4 تا PA7 در CubeMX است.



فصل ۱۱: مدیریت همزمان وظایف؛ تقابل CMSIS و Native FreeRTOS

تا کنون تمام برنامه‌های ما در یک حلقه ابدی (1) `while` اجرا می‌شدند که باعث می‌شد پردازنده در یک زمان فقط بتواند یک کار را انجام دهد. سیستم‌عامل **FreeRTOS** به ما اجازه می‌دهد برنامه را به بخش‌های مستقلی به نام تسک (**Task**) تقسیم کنیم که به طور ظاهراً همزمان اجرا می‌شوند. (چون `free rtos` روی پروتئوس درست اجرا نمیشه نیازه که سخت افزار `stm32` رو داشته باشید برای تست)

۱. تعاریف اولیه و دستگیره‌ها (Handles)

برای اینکه سیستم‌عامل بتواند یک تسک را مدیریت، متوقف یا اجرا کند، به یک متغیر برای شناسایی آن نیاز دارد که به آن **Handle** می‌گوییم.

- در سبک **CMSIS-RTOS** از نوع داده `osThreadId` استفاده می‌شود که یک شناسه انتزاعی است.
- در سبک **Native FreeRTOS** از نوع داده `TaskHandle_t` استفاده می‌شود که مستقیماً به بلوک کنترل تسک اشاره دارد.

۲. ساخت و پیکربندی تسک‌ها (Task Creation)

تسک‌ها باید قبل از شروع به کار سیستم‌عامل تعریف و ساخته شوند.

- در سبک **CMSIS-RTOS** ابتدا با ماکروی `osThreadDef` ویژگی‌های تسک (نام، اولویت، تعداد نمونه و اندازه پشته) تعریف شده و سپس با `osThreadCreate` ساخته می‌شود.
- در سبک **Native FreeRTOS** تمام پارامترها مستقیماً به تابع `xTaskCreate` پاس داده می‌شوند. این تابع منعطف‌تر است و خروجی آن مشخص می‌کند که آیا حافظه کافی برای ساخت تسک وجود داشته است یا خیر.

۳. شروع زمان‌بندی (Starting the Scheduler)

- پس از ساخت تسک‌ها، باید واحد مدیریت زمان یا **Scheduler** را فعال کنیم.
- در سبک **CMSIS-RTOS** از تابع `osKernelStart()` استفاده می‌شود.
 - در سبک **Native FreeRTOS** از تابع `vTaskStartScheduler()` ستفاده می‌شود.
- پس از اجرای این دستور، پردازنده دیگر به حلقه `while(1)` اصلی باز نمی‌گردد و مدیریت کلاً به دست سیستم‌عامل می‌افتد.

۴. کد کامل پروژه LED Blinking با دو سبک

این کد دو LED متصل به پایه‌های **PB12** و **PB13** را با فرکانس‌های متفاوت کنترل می‌کند.

```
1. /* Includes -----*/
2. #include "main.h"
3. #include "cmsis_os.h" // CMSIS-RTOS wrapper
4. #include <stdio.h>
5.
6. /* Private variables -----*/
7. // Handles for Tasks (Native FreeRTOS Style)
8. TaskHandle_t led1TaskHandle;
9. TaskHandle_t led2TaskHandle;
10.
11. // Handles for Tasks (CMSIS-RTOS Style - Commented)
12. // osThreadId led1TaskHandle_cmsis;
13.
14. /* Function Prototypes -----*/
15. void led1Task(void * argument);
16. void led2Task(void * argument);
17.
18. int main(void)
19. {
20.     HAL_Init();
```

```

21.  SystemClock_Config();
22.  MX_GPIO_Init();
23.
24.  /* --- Task Creation Section --- */
25.
26.  // STYLE 1: Native FreeRTOS (Recommended for performance)
27.  // Parameters: Function, Name, Stack Size, Argument, Priority, Handle
28.  if(xTaskCreate(led1Task, "LED1_Task", 128, NULL, tskIDLE_PRIORITY + 1,
&led1TaskHandle) != pdPASS)
29.  {
30.      while(1); // Error: Not enough memory (Heap)
31.  }
32.
33.  // STYLE 2: CMSIS-RTOS (Standard for portability)
34.  /*
35.  osThreadDef(task2, led2Task, osPriorityNormal, 0, 128);
36.  led1TaskHandle_cmsis = osThreadCreate(osThread(task2), NULL);
37.  */
38.
39.  // Creating second task using Native FreeRTOS
40.  xTaskCreate(led2Task, "LED2_Task", 128, NULL, tskIDLE_PRIORITY + 1,
&led2TaskHandle);
41.
42.  /* --- Start Scheduler --- */
43.
44.  // STYLE 1: Native FreeRTOS
45.  vTaskStartScheduler();
46.
47.  // STYLE 2: CMSIS-RTOS
48.  // osKernelStart();
49.
50.  while (1)
51.  {
52.      // Program never reaches here if Scheduler starts successfully
53.  }
54. }
55.
56. /* --- Task Implementation --- */
57.
58. // LED1 Task: Toggles every 300ms
59. void led1Task(void * argument)
60. {
61.     while(1)
62.     {
63.         HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_12);
64.
65.         // Delay Style 1: Native FreeRTOS (Using Ticks)
66.         vTaskDelay( 300 / portTICK_PERIOD_MS );
67.
68.         // Delay Style 2: CMSIS-RTOS
69.         // osDelay(300);
70.     }
71. }
72.
73. // LED2 Task: Toggles every 350ms

```

```

74. void led2Task(void * argument)
75. {
76.     while(1)
77.     {
78.         HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_13);
79.
80.         // Delay Style 1: Native FreeRTOS (Using helper macro)
81.         vTaskDelay( pdMS_TO_TICKS(350) );
82.
83.         // Delay Style 2: CMSIS-RTOS
84.         // osDelay(350);
85.
86.     }
87. }

```

۵. تحلیل فنی تاخیرها (vTaskDelay vs osDelay)

در برنامه‌نویسی بدون سیستم‌عامل، از HAL_Delay استفاده می‌کردیم که پردازنده را در یک حلقه خالی معطل می‌کرد. اما در RTOS:

- **vTaskDelay:** تسک را به وضعیت **Blocked** می‌برد. در این مدت، سیستم‌عامل متوجه می‌شود که این تسک کاری برای انجام دادن ندارد، بنابراین پردازنده را به تسک‌های دیگر می‌دهد.
- **portTICK_PERIOD_MS:** این مقدار به ما کمک می‌کند تا زمان را از میلی‌ثانیه به "تیک‌های سیستم" تبدیل کنیم. اگر کلاک سیستم تغییر کند، این ماکرو تضمین می‌کند که زمان تاخیر همچنان دقیق باقی بماند.
- **pdMS_TO_TICKS:** یک ماکروی خواناتر برای تبدیل میلی‌ثانیه به تیک است که استفاده از آن در پروژه‌های حرفه‌ای توصیه می‌شود.

فصل ۱۲: از کدنویسی تا واقعیت؛ اجرای پروژه روی برد

خب، خسته نباشی! الان کدی که نوشتی آماده‌ست و منتظره که جون بگیره. تا الان توی دنیای نرم‌افزار بودیم، ولی حالا وقتشه آستین‌ها رو بالا بزنیم و بریم سراغ سخت‌افزار. توی این فصل قدم‌به‌قدم یاد می‌گیریم چطور این کد رو کامپایل کنیم، چه بردی بخریم و چطور برنامه‌مون رو منتقل کنیم روی آی‌سی.

قدم اول: کامپایل کردن (مطمئن شو کد سالمه)

قبل از اینکه به سیم و سخت‌افزار دست بزنیم، باید مطمئن بشیم کدی که توی `STM32CubeIDE` نوشتی، ایراد نگارشی نداره و کامپیوتر می‌فهمه چی نوشتی.

۱. توی محیط `STM32CubeIDE`، اون بالا سمت چپ یه آیکون شبیه چکش (`Hammer`)

هست. روش کلیک کن.

۲. پایین صفحه توی بخش `Console` شروع می‌کنه به نوشتن یه سری متن. صبر کن تا تموم بشه.

۳. اگر آخرش نوشت `Build Finished. 0 errors`، یعنی دمت گرم! کدت سالمه و تبدیل شده به فایل `Hex` زبان ماشین.

۴. اگر ارور داد، باید بری خطی که قرمز شده رو چک کنی ببینی سمی‌کالنی چیزی جا ننداخته باشی.

قدم دوم: چی بخریم؟ (انتخاب سلاح!)

برای اجرای این پروژه، تو به یک "مغز" (برد میکروکنترلر) و یک "رابط" (پروگرامر) نیاز داری. دو تا گزینه محبوب داری:

گزینه ۱: برد دیسکاوری - (`Discovery`) راحت ولی گرون

این بردها، بردهای رسمی شرکت `ST` هستن.

- **خویش چیه؟** روی خودشون پروگرمر دارن. یعنی فقط با یه کابل USB وصل می‌شن به کامپیوتر و تمام.
- **بدیش چیه؟** گرون و بزرگ. معمولاً برای یادگیری عالی‌ان ولی همیشه گذاشتشون توی یه محصول نهایی کوچیک.

گزینه ۲: برد Black Pill یا Blue Pill پیشنهادی و اقتصادی

این‌ها بردهای کوچیک و سیاه‌رنگی هستن که خیلی محبوبن.

- **خویش چیه؟** خیلی ارزون، جمع‌وجورن و قدرتمند (مخصوصاً مدل‌های F401 یا F411).
- **بدیش چیه؟** پروگرمر داخلی ندارن. یعنی باید یه قطعه جداگانه به اسم **ST-Link V2** هم براش بخری.

پیشنهاد من: به برد **Black Pill** مدل STM32F411 عالی‌ه بخر به همراه یه دانگل **ST-Link V2**. هم ارزون‌تر در میاد، هم یاد می‌گیری چطور سخت‌افزارها رو بهم وصل کنی.

قدم سوم: سیم‌کشی و اتصال پروگرمر

اگر برد دیسکاواری خریدی که هیچی، کابل USB رو بزن و برو مرحله بعد. اما اگر **Black Pill** و **ST-Link** خریدی، باید این‌ها رو با ۴ تا سیم مادگی-مادگی بهم وصل کنی. اینجا رو خیلی دقت کن، چون اشتباه کردن می‌تونه میکرو رو بسوزونه!

توضیحات (مهم!)	پایه روی Black Pill	پایه روی ST-Link
منفی مشترک (حتماً اول اینو وصل کن)	GND	GND
کلاک ارتباطی	SWCLK یا PA14	SWCLK
دیتای ارتباطی	SWDIO یا PA13	SWDIO
تغذیه مثبت	3.3V	3.3V

هشدار مرگبار: روی پروگرمر یه پایه ۵ ولت هم هست. هرگز، هرگز و هرگز پایه ۵ ولت پروگرمر رو به پایه ۳.۳ ولت برد وصل نکن. میکروکنترلر در جا می‌سوزه! همیشه ۳.۳ رو به ۳.۳ وصل کن.

قدم چهارم: ریختن برنامه روی برد (Program)

حالا که سیم‌ها وصله و چراغ قرمز روی برد روشن شده:

۱. برگرد توی **STM32CubeIDE**.
۲. اون بالا به دکمه **سبز رنگ Play** (شبيه دکمه پخش موزیک) هست. روش کلیک کن.
۳. به پنجره باز میشه به اسم **Edit Configuration** معمولاً لازم نیست چیزی رو تغییر بدی، فقط **OK** رو بزنی.

۴. اگر همه چی درست باشه، توی کنسول می‌نویسه **Download verified successfully** و برنامه روی برد اجرا میشه.

نکته: اگر دوست داری کدت رو خط‌به‌خط تست کنی، به جای دکمه **Play**، روی دکمه **(Debug)** کلیک کن.

نکات کنکوری و پایانی

- اگر ارور **"Target Not Found"** داد:

- اول سیم‌کشی رو چک کن **GND** و **V ۳.۳** شل نباشن.
 - ممکنه درایور **ST-Link** روی ویندوزت نصب نباشه. حتماً درایورش رو نصب کن.
 - گاهی وقتاً باید دکمه **"Reset"** روی برد رو نگه داری، توی نرم‌افزار دکمه **Play** رو بزنی و به محض اینکه دیدی داره تلاش می‌کنه وصل بشه، دکمه **Reset** رو ول کنی (به این میگن روش **Connect under reset**).
 - **چراغ‌ها:** روی **ST-Link** معمولاً به چراغ چشمک‌زن هست. وقتی داره برنامه رو می‌ریزه، این چراغ تندتند چشمک می‌زنه و بعد ثابت میشه.
- تموم شد! الان تو به سخت‌افزار واقعی داری که کدی که نوشتی داره توش نفس می‌کشه. از اینجا به بعد دیگه خلاقیت خودته که چه ایده‌هایی رو با این چیپ‌های قدرتمند پیاده کنی. موفق باشی!

ماجرای اون جامپرهای زرد رنگ **BOOT0** و **BOOT1**

اگه برد **Blue Pill** یا **Black Pill** خریده باشی، احتمالاً دو تا جامپر کوچیک زرد رنگ (یا گاهی سیاه) روش می‌بینی که کنارشون نوشته **BOOT0** و **BOOT1**. وضعیت این جامپرها تعیین می‌کنه که وقتی برد روشن میشه، میکروکنترلر بره سراغ اجرای کد تو، یا بره تو حالت‌های خاص دیگه.

برای اینکه گیج نشی، این قانون کلی رو یادت باشه:

۱. حالت عادی و همیشگی (اجرای برنامه)

- وضعیت: هر دو جامپر BOOT0 و BOOT1 باید روی عدد 0 باشن یعنی وصل باشن به سمت منفی. GND.

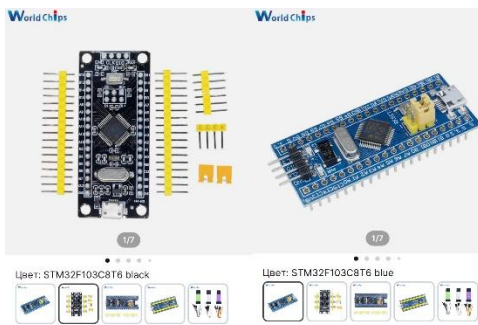
- توضیح: توی این حالت، به محض اینکه به برد برق بدی، کدی که نوشتی اجرا میشه. برای پروگرام کردن با ST-Link هم معمولاً همین حالت جواب میده و نیازی نیست بهش دست بزنی.

۲. حالت اضطراری (وقتی برد پروگرام نمیشه)

- وضعیت: جامپر BOOT0 رو بزار روی 1 (وصل کن به ۳.۳ ولت) و BOOT1 همون 0 بمونه.

- توضیح: گاهی اوقات کدی می نویسی که باعث میشه میکرو هنگ کنه یا پایه‌های پروگرامر رو غیرفعال کنه و دیگه ST-Link برد رو نمی شناسه. اینجور وقتا جامپر رو بزار روی ۱، دکمه ریست رو بزنی و پروگرام کن. بعد از اینکه پروگرام شد، حتماً دوباره برش گردون به ۰ تا برنامه جدیدت اجرا بشه.

خلاصه: همیشه بذار روی 0-0 بمونه. اگه دیدی ST-Link ارور میده و وصل نمیشه، یه بار بذار روی 1-0، پروگرام کن و دوباره برگردون سر جاش.



STMicroelectronics